Minishell

Generated by Doxygen 1.9.1

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

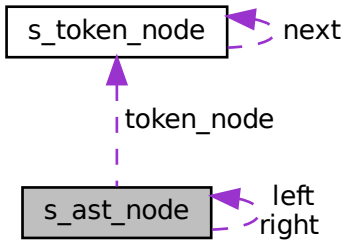
Chapter 3

Class Documentation

3.1 s_ast_node Struct Reference

`#include <ast.h>`

Collaboration diagram for s_ast_node:



Public Attributes

- t_token_type type
- t_token_node * token_node
- struct s_ast_node * left
- struct s_ast_node * right

3.1.1 Member Data Documentation

3.1.1.1 left

```
struct s_ast_node* s_ast_node::left
```

3.1.1.2 right

```
struct s_ast_node* s_ast_node::right
```

3.1.1.3 token_node

```
t_token_node* s_ast_node::token_node
```

3.1.1.4 type

```
t_token_type s_ast_node::type
```

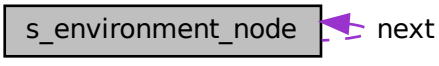
The documentation for this struct was generated from the following file:

- inc/ast.h

3.2 s_environment_node Struct Reference

```
#include <environment.h>
```

Collaboration diagram for s_environment_node:



Public Attributes

- char * key
- char * value
- struct s_environment_node * next
- t_bool is_private

3.2.1 Member Data Documentation

3.2.1.1 is_private

```
t_bool s_environment_node::is_private
```

3.2.1.2 key

```
char* s_environment_node::key
```

3.2.1.3 next

```
struct s_environment_node* s_environment_node::next
```

3.2.1.4 value

```
char* s_environment_node::value
```

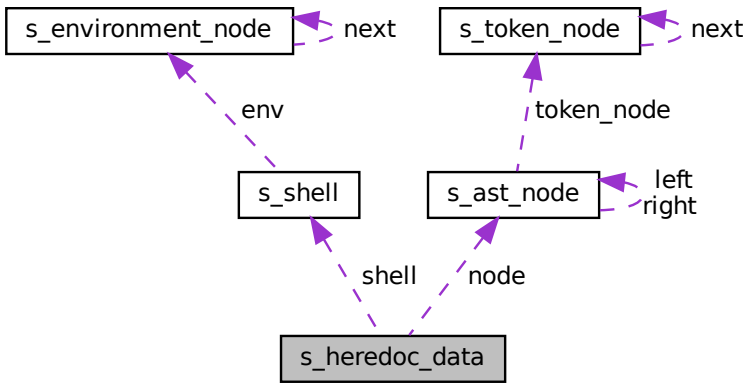
The documentation for this struct was generated from the following file:

- inc/environment.h

3.3 s_heredoc_data Struct Reference

```
#include <execute.h>
```

Collaboration diagram for s_heredoc_data:



Public Attributes

- pid_t pid
- t_ast_node * node
- t_shell * shell
- int pipe_fd [2]
- int original_stdin

3.3.1 Member Data Documentation

3.3.1.1 node

```
t_ast_node* s_heredoc_data::node
```

3.3.1.2 original_stdin

```
int s_heredoc_data::original_stdin
```

3.3.1.3 pid

```
pid_t s_heredoc_data::pid
```

3.3.1.4 pipe_fd

```
int s_heredoc_data::pipe_fd[2]
```

3.3.1.5 shell

```
t_shell* s_heredoc_data::shell
```

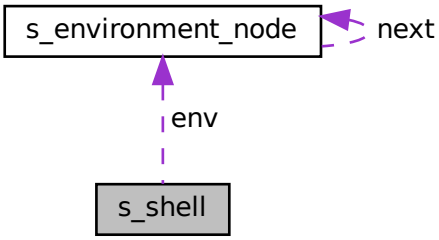
The documentation for this struct was generated from the following file:

- inc/execute.h

3.4 s_shell Struct Reference

`#include <minishell.h>`

Collaboration diagram for s_shell:



Public Attributes

- t_environment_node * env
- int exit_code
- const char ** envp
- t_bool top_level_redir_out_enabled
- t_bool top_level_redir_in_enabled

3.4.1 Member Data Documentation

3.4.1.1 env

```
t_environment_node* s_shell::env
```

3.4.1.2 envp

```
const char** s_shell::envp
```

3.4.1.3 exit_code

```
int s_shell::exit_code
```

3.4.1.4 top_level_redir_in_enabled

```
t_bool s_shell::top_level_redir_in_enabled
```

3.4.1.5 top_level_redir_out_enabled

```
t_bool s_shell::top_level_redir_out_enabled
```

The documentation for this struct was generated from the following file:

- inc/minishell.h

3.5 s_token Struct Reference

Represents a token with a type and associated arguments.

```
#include <tokens.h>
```

3.5.1 Detailed Description

Represents a token with a type and associated arguments.

- s_token::type The type of the token, represented by t_token_type.
- s_token::args A pointer to an array of strings (char**) representing the arguments associated with the token, null terminated.

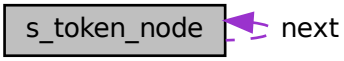
The documentation for this struct was generated from the following file:

- inc/tokens.h

3.6 s_token_node Struct Reference

```
#include <tokens.h>
```

Collaboration diagram for s_token_node:



Public Attributes

- t_token_type type
- char ** args
- struct s_token_node * next

3.6.1 Member Data Documentation

3.6.1.1 args

```
char** s_token_node::args
```

3.6.1.2 next

```
struct s_token_node* s_token_node::next
```

3.6.1.3 type

```
t_token_type s_token_node::type
```

The documentation for this struct was generated from the following file:

- inc/tokens.h

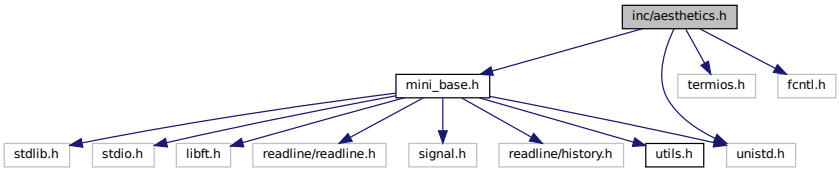
Chapter 4

File Documentation

4.1 inc/aesthetics.h File Reference

```
#include <mini_base.h>
#include <termios.h>
#include <fcntl.h>
#include <unistd.h>
```

Include dependency graph for aesthetics.h:



This graph shows which files directly or indirectly include this file:

Macros

- #define HIDE_CURSOR "\033[?25l"
- #define SHOW_CURSOR "\033[?25h"
- #define RED "\033[31m"
- #define GREEN "\033[32m"
- #define YELLOW "\033[33m"
- #define BLUE "\033[34m"
- #define MAGENTA "\033[35m"
- #define CYAN "\033[36m"
- #define RESET "\033[0m"
- #define INTRO_SECONDS 10
- #define SECOND_FROM_MICRO 1000000
- #define SPINNER "-\\|/"

4.1.1 Macro Definition Documentation

4.1.1.1 BLUE

```
#define BLUE "\033[34m"
```

4.1.1.2 CYAN

```
#define CYAN "\033[36m"
```

4.1.1.3 GREEN

```
#define GREEN "\033[32m"
```

4.1.1.4 HIDE_CURSOR

```
#define HIDE_CURSOR "\033[?25l"
```

4.1.1.5 INTRO_SECONDS

```
#define INTRO_SECONDS 10
```

4.1.1.6 MAGENTA

```
#define MAGENTA "\033[35m"
```

4.1.1.7 RED

```
#define RED "\033[31m"
```

4.1.1.8 RESET

```
#define RESET "\033[0m"
```

4.1.1.9 SECOND_FROM_MICRO

```
#define SECOND_FROM_MICRO 1000000
```

4.1.1.10 SHOW_CURSOR

```
#define SHOW_CURSOR "\033[?25h"
```

4.1.1.11 SPINNER

```
#define SPINNER "-\\|/"
```

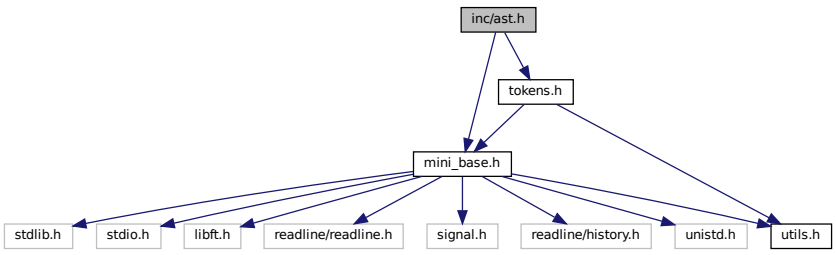
4.1.1.12 YELLOW

```
#define YELLOW "\033[33m"
```

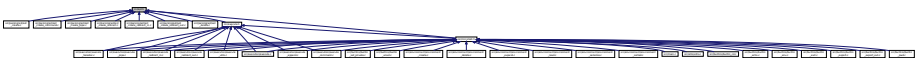
4.2 inc/ast.h File Reference

```
#include <mini_base.h>
#include <tokens.h>
```

Include dependency graph for ast.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct s_ast_node

Typedefs

- typedef struct s_ast_node t_ast_node

Functions

- t_ast_node * ast_create_command (t_token_node **list)
- t_ast_node * ast_create_node (t_token_node *token_node)

  Creates a new AST node.
- t_ast_node * ast_create_pipe (t_token_node **list)
- t_ast_node * ast_create_redirect_in_heredoc (t_token_node **list)
- t_ast_node * ast_create_redirect_out_append (t_token_node **list)
- t_ast_node * ast_create_redirect (t_token_node **list)
- t_ast_node * ast_create (t_token_node **list)

  Creates an abstract syntax tree (AST) from a list of tokens.
- void ast_delete (t_ast_node *node)

  Recursively deletes an Abstract Syntax Tree (AST) node and its children.
- void ast_print (t_ast_node *node)

4.2.1 Typedef Documentation

4.2.1.1 t_ast_node

```
typedef struct s_ast_node t_ast_node
```

4.2.2 Function Documentation

4.2.2.1 ast_create()

```
t_ast_node* ast_create (
            t_token_node ** list )
```

Creates an abstract syntax tree (AST) from a list of tokens.

This function takes a list of tokens and constructs an AST node from it. If the list is empty or NULL, the function returns NULL. Otherwise, it delegates the creation of the AST to the `ast_create_pipe` function.

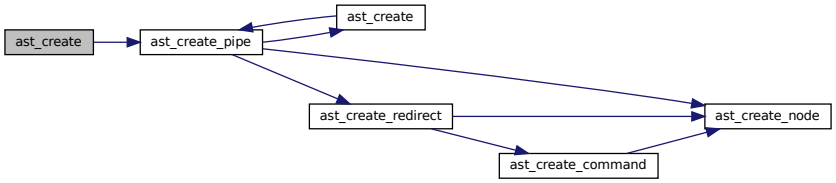
Parameters

list	A pointer to the list of tokens to be parsed into an AST.

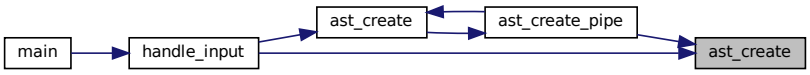
Returns

A pointer to the created AST node, or NULL if the list is empty or NULL.

Here is the call graph for this function:



Here is the caller graph for this function:



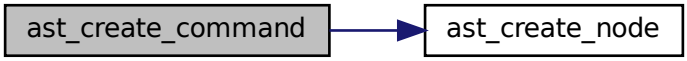
4.2.2.2 ast_create_command()

```
t_ast_node* ast_create_command (
            t_token_node ** list )
```

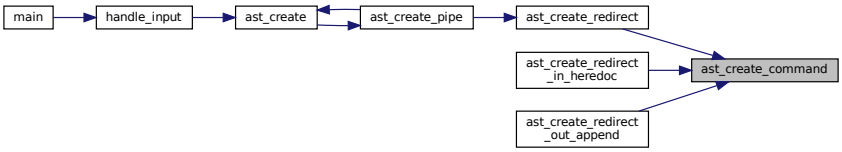
ast_parse_command - Parses a command from a token list and creates an AST node. @list: A double pointer to the head of the token list.

This function checks if the provided token list is valid and if the head of the list is of type TOKEN_STRING. If so, it creates an AST node from the head token, advances the list to the next token, and returns the created AST node. If the list is invalid or the head token is not of type TOKEN_STRING, it returns NULL.

Return: A pointer to the created AST node, or NULL if the list is invalid or the head token is not of type TOKEN_↩ STRING. Here is the call graph for this function:



Here is the caller graph for this function:



4.2.2.3 ast_create_node()

```
t_ast_node* ast_create_node (
            t_token_node * token_node )
```

Creates a new AST node.

This function allocates memory for a new `t_ast_node` structure and initializes its fields. The `type` parameter specifies the type of the node, while the `value` parameter specifies the value associated with the node. If `value` is `NULL`, the `value` field of the node will be set to `NULL`.

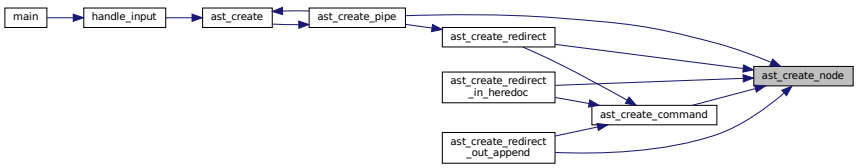
Parameters

type	The type of the AST node.
value	The value associated with the AST node.

Returns

A pointer to the newly created `t_ast_node` structure, or `NULL` if memory allocation fails.

Here is the caller graph for this function:



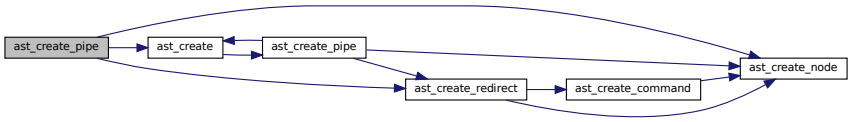
4.2.2.4 ast_create_pipe()

```
t_ast_node* ast_create_pipe (
            t_token_node ** list )
```

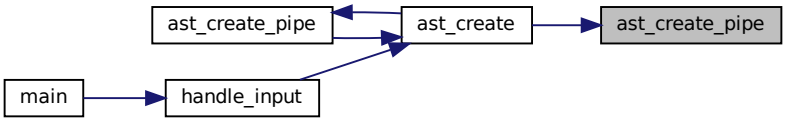
ast_create_pipe - Creates an AST node for a pipe operation. @list: A pointer to the list of token nodes.

This function creates an abstract syntax tree (AST) node for a pipe operation by parsing the token list. It first creates the left node by calling ast_create_redirect. Then, it iterates through the token list to find pipe tokens (TOKEN_↩ PIPE). For each pipe token found, it creates a right node by calling ast_create, and then creates a pipe node with the current token. The left and right nodes are assigned to the pipe node, and the left node is updated to the pipe node. The function returns the final left node, which represents the root of the pipe operation AST.

Return: A pointer to the root AST node of the pipe operation. Here is the call graph for this function:



Here is the caller graph for this function:



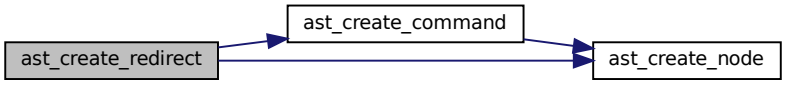
4.2.2.5 ast_create_redirect()

```
t_ast_node* ast_create_redirect (
            t_token_node ** list )
```

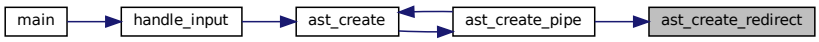
ast_create_redirect - Parses a list of tokens to create an abstract syntax tree (AST) node representing a redirection. @list: A pointer to the list of tokens to be parsed.

This function parses a list of tokens to create an AST node that represents a redirection operation. It first parses the left-hand side command, then iterates through the list of tokens to check for redirection operators (heredoc, redirect in, append, redirect out). For each redirection operator found, it creates a new AST node, sets the left and right children, and updates the list pointer to the next token. The function returns the root of the created AST subtree.

Return: A pointer to the root AST node representing the redirection. Here is the call graph for this function:



Here is the caller graph for this function:



4.2.2.6 ast_create_redirect_in_heredoc()

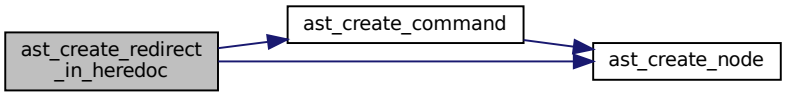
```
t_ast_node* ast_create_redirect_in_heredoc (
            t_token_node ** list )
```

ast_create_redirect_in_heredoc - Parses a list of tokens to create an AST node for input redirection and heredoc.

@list: Double pointer to the head of the token list.

This function processes a list of tokens to create an abstract syntax tree (AST) node for input redirection and heredoc. It starts by creating a command node from the token list and then iterates through the list to handle heredoc and input redirection tokens. For each heredoc or input redirection token, it creates a new AST node, sets the left and right children, and updates the left node to the newly created node. The function returns the final left node, which represents the root of the constructed AST for input redirection and heredoc.

Return: A pointer to the root of the constructed AST node. Here is the call graph for this function:



4.2.2.7 ast_create_redirect_out_append()

```
t_ast_node* ast_create_redirect_out_append (
            t_token_node ** list )
```

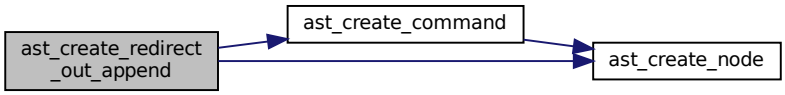
ast_create_redirect_out_append - Parses a list of tokens to create an AST node for redirect out or append operations.

@list: A pointer to the head of the token list.

This function processes a list of tokens to create an abstract syntax tree (AST) node representing redirect out (>) or append (>>) operations. It starts by creating a command node from the token list and then iterates through the list to handle consecutive redirect out or append tokens. For each such token, it creates a new AST node, sets the

left and right children, and updates the left child to the newly created node. The function returns the final AST node representing the parsed redirect out or append operations.

Return: A pointer to the root of the created AST node. Here is the call graph for this function:



4.2.2.8 ast_delete()

```
void ast_delete (
            t_ast_node * node )
```

Recursively deletes an Abstract Syntax Tree (AST) node and its children.

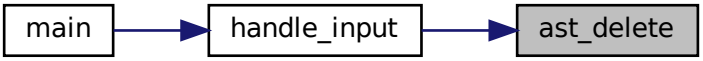
This function takes a pointer to an AST node and recursively deletes its left and right children, as well as the token associated with the node. Finally, it frees the memory allocated for the node itself.

Parameters

node	A pointer to the AST node to be deleted. If the node is NULL, the function returns immediately.

Here is the call graph for this function:

Here is the caller graph for this function:



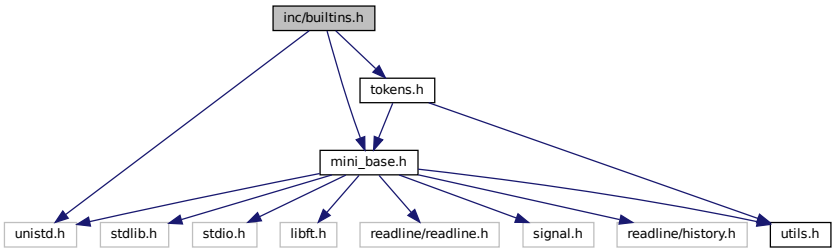
4.2.2.9 ast_print()

```
void ast_print (
            t_ast_node * node )
```

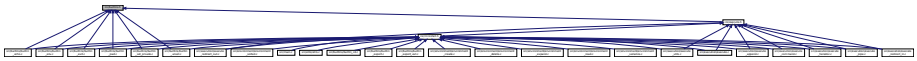
4.3 inc/builtins.h File Reference

```
#include <unistd.h>
#include <mini_base.h>
#include <tokens.h>
```

Include dependency graph for builtins.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define BUILTIN_EXIT_TOO_MANY_ARGS -1
- #define BUILTIN_EXIT_NON_NUMERIC_ARG 2
- #define BUILTIN_EXIT_OK 0

Typedefs

- typedef struct s_shell t_shell
- typedef struct s_ast_node t_ast_node

Functions

- char * builtin_pwd (void)

  Retrieves the current working directory.
- void execute_builtin_pwd (t_shell *shell)

  Executes the built-in pwd command.
- void builtin_env (t_shell *shell)

  Prints the current environment variables.
- void builtin_unset (t_shell *shell, t_ast_node *node)

  Unsets environment variables in the shell.
- void builtin_set_private (t_shell *shell, t_ast_node *node)

  Sets a private environment variable in the shell.
- void builtin_export (t_shell *shell, t_ast_node *node)

  Handles the export builtin command in the shell.
- t_bool builtin_export_export_key_value (t_shell *shell, t_ast_node *node)

  Processes the export command by handling multiple key-value pairs.
- t_bool builtin_exit (t_shell *shell, t_ast_node **node_ptr, char **line_ptr, t_token_node ***list_ptr)

  Handles the execution of the `exit` built-in command.
- void builtin_echo (t_shell *shell, t_ast_node *node)

  Implements the `echo` built-in command for the minishell.
- void builtin_cd (t_shell *shell, char **args)

  Changes the current working directory of the shell.

4.3.1 Macro Definition Documentation

4.3.1.1 BUILTIN_EXIT_NON_NUMERIC_ARG

```
#define BUILTIN_EXIT_NON_NUMERIC_ARG 2
```

4.3.1.2 BUILTIN_EXIT_OK

```
#define BUILTIN_EXIT_OK 0
```

4.3.1.3 BUILTIN_EXIT_TOO_MANY_ARGS

```
#define BUILTIN_EXIT_TOO_MANY_ARGS -1
```

4.3.2 Typedef Documentation

4.3.2.1 t_ast_node

```
typedef struct s_ast_node t_ast_node
```

4.3.2.2 t_shell

```
typedef struct s_shell t_shell
```

4.3.3 Function Documentation

4.3.3.1 builtin_cd()

```
void builtin_cd (
            t_shell * shell,
            char ** args )
```

Changes the current working directory of the shell.

This function implements the `cd` built-in command, allowing users to navigate the filesystem. It supports:

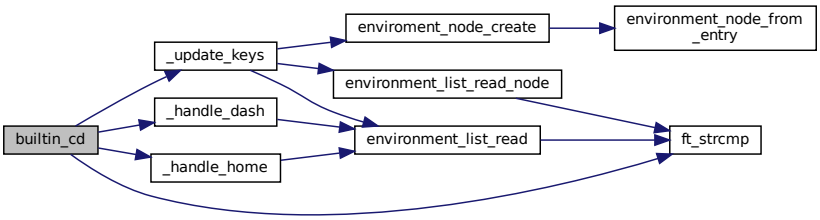
- `cd` or `cd ~` to move to the home directory.
- `cd -` to move to the previous working directory.
- `cd <directory>` to move to the specified directory.

If the directory change is successful, it updates the `PWD` and `OLDPWD` environment variables. If it fails, an error message is printed.

Parameters

shell	A pointer to the shell structure.
args	An array of arguments passed to the `cd` command.

Here is the call graph for this function:



4.3.3.2 builtin_echo()

```
void builtin_echo (
            t_shell * shell,
            t_ast_node * node )
```

Implements the `echo` built-in command for the minishell.

This function mimics the behavior of the UNIX `echo` command, supporting the `-n` option to suppress the trailing newline. It expands environment variables within arguments and prints the result to standard output.

Parameters

shell	The shell structure containing environment variables and state.
node	The AST node representing the `echo` command and its arguments.

4.3.3.3 builtin_env()

```
void builtin_env (
            t_shell * shell )
```

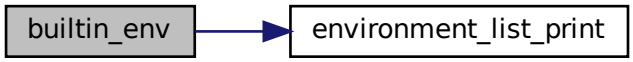
Prints the current environment variables.

This function prints all the environment variables stored in the shell's environment list. It also sets the shell's exit code to 0 upon completion.

Parameters

shell	A pointer to the shell structure containing the environment list.

Here is the call graph for this function:



4.3.3.4 builtin_exit()

```
t_bool builtin_exit (
            t_shell * shell,
            t_ast_node ** node_ptr,
            char ** line_ptr,
            t_token_node *** list_ptr )
```

Handles the execution of the `exit` built-in command.

This function checks if the `exit` command is properly formatted and executes it, freeing necessary resources before terminating the shell. The function ensures:

- The command is correctly formatted with a single `exit` keyword.
- Argument validity is checked, allowing a numeric exit code.
- The shell's exit code is set appropriately.
- Memory cleanup is performed before exiting.

Parameters

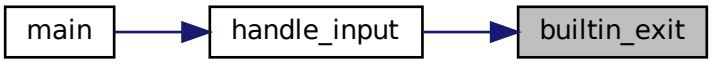
shell	A pointer to the shell structure.
node_ptr	A double pointer to the AST node containing the command.
line_ptr	A pointer to the command line input to be freed.
list_ptr	A pointer to the token list to be freed.

Returns

true if the `exit` command was executed, false otherwise.

Here is the call graph for this function:

Here is the caller graph for this function:



4.3.3.5 builtin_export()

```
void builtin_export (
            t_shell * shell,
            t_ast_node * node )
```

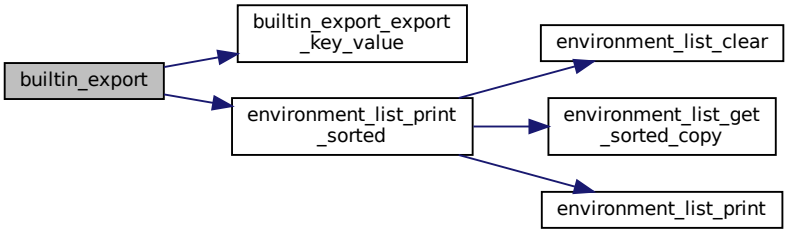
Handles the export builtin command in the shell.

This function processes the export command, which is used to set environment variables. If no arguments are provided, it prints the environment variables in sorted order. If arguments are provided, it attempts to set the specified environment variables.

Parameters

shell	A pointer to the shell structure containing the environment.
node	A pointer to the AST node representing the export command and its arguments.

Here is the call graph for this function:



4.3.3.6 builtin_export_export_key_value()

```
t_bool builtin_export_export_key_value (
            t_shell * shell,
            t_ast_node * node )
```

Processes the export command by handling multiple key-value pairs.

This function iterates through the provided arguments, processing each as either a single key or a key-value pair. It updates the environment variables accordingly.

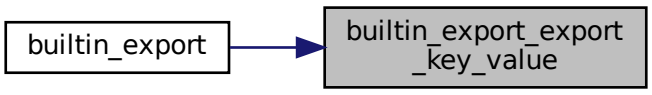
Parameters

shell	A pointer to the shell structure.
node	The AST node containing the command and its arguments.

Returns

true if an error occurs during processing, false otherwise.

Here is the caller graph for this function:



4.3.3.7 builtin_pwd()

```
char* builtin_pwd (
            void  )
```

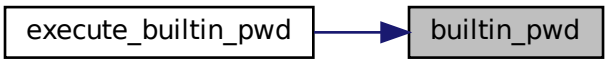
Retrieves the current working directory.

This function uses the getcwd() function to obtain the current working directory and returns it as a dynamically allocated string. The caller is responsible for freeing the allocated memory.

Returns

A pointer to the current working directory string, or NULL if an error occurs.

Here is the caller graph for this function:



4.3.3.8 builtin_set_private()

```
void builtin_set_private (
            t_shell * shell,
            t_ast_node * node )
```

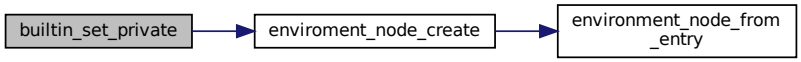
Sets a private environment variable in the shell.

This function takes a shell structure and an AST node, and sets a private environment variable based on the arguments provided in the AST node. If the required arguments are not present, it sets the shell's exit code to 1 and returns. Otherwise, it creates a new environment entry and updates the shell's environment.

Parameters

shell	A pointer to the shell structure.
node	A pointer to the AST node containing the arguments.

Here is the call graph for this function:



4.3.3.9 builtin_unset()

```
void builtin_unset (
            t_shell * shell,
            t_ast_node * node )
```

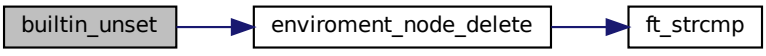
Unsets environment variables in the shell.

This function removes environment variables specified in the arguments from the shell's environment. If no arguments are provided, it prints an error message indicating that not enough arguments were given.

Parameters

shell	A pointer to the shell structure.
node	A pointer to the AST node containing the command and its arguments.

Here is the call graph for this function:



4.3.3.10 execute_builtin_pwd()

```
void execute_builtin_pwd (
            t_shell * shell )
```

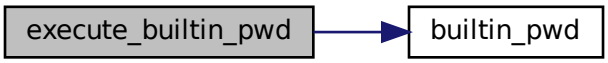
Executes the built-in pwd command.

This function retrieves the current working directory using the builtin_pwd() function. If the directory is successfully retrieved, it prints the directory path to the standard output and sets the shell's exit code to 0. If the directory retrieval fails, it prints an error message using perror and sets the shell's exit code to 1.

Parameters

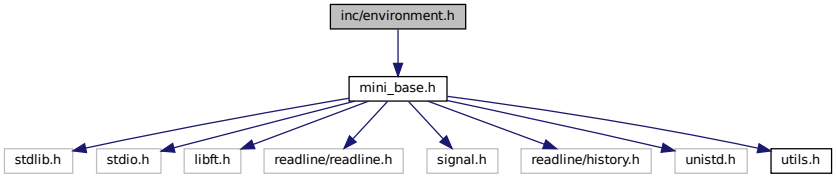
shell	A pointer to the shell structure containing the exit code.

Here is the call graph for this function:

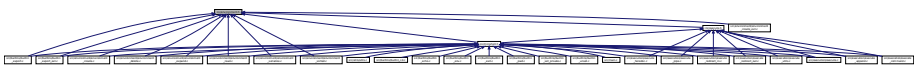


4.4 inc/environment.h File Reference

`#include <mini_base.h>`
Include dependency graph for environment.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct s_environment_node

Typedefs

- typedef struct s_shell t_shell
- typedef struct s_environment_node t_environment_node

Functions

- void environment_list_clear (t_shell *shell)

  Clears all nodes in the shell's environment list.
- char * environment_list_read (const char *key, t_shell *shell)

  Reads the value associated with a given key from the environment list.
- void enviroment_node_delete (const char *key, t_shell *shell)

  Deletes an environment node with a specified key from the shell's environment list.
- t_environment_node * environment_list_initialize (const char **envp)

  Initializes an environment list from an array of environment variable strings.
- t_environment_node * enviroment_node_create (const char *entry, t_shell *shell, t_bool is_private)

  Creates an environment node from an entry string and adds it to the shell's environment list.
- char * env_value_expand (t_shell *shell, char *key)

  Expands the value of an environment variable based on the given key.
- char ** environment_serialize (t_shell *shell)

  Serializes the environment variables from the shell into an array of strings.
- void environment_serialized_list_clear (char **list)

  Frees a list of strings and the list itself.
- void environment_list_print (t_shell *shell, t_bool is_export)

  Prints the environment variables in the shell.
- t_environment_node * environment_node_from_entry (const char *entry, t_bool is_private)

  Creates a new environment node from an entry string.
- void environment_list_print_sorted (t_shell *shell)

  Prints the environment variables in sorted order.
- t_environment_node * environment_list_get_sorted_copy (t_environment_node *original)

  Creates a sorted copy of the environment list.
- t_environment_node * environment_list_read_node (const char *key, t_shell *shell)

  Reads a node from the environment list based on the given key.

4.4.1 Typedef Documentation

4.4.1.1 t_environment_node

```
typedef struct s_environment_node t_environment_node
```

4.4.1.2 t_shell

```
typedef struct s_shell t_shell
```

4.4.2 Function Documentation

4.4.2.1 env_value_expand()

```
char* env_value_expand (
            t_shell * shell,
            char * key )
```

Expands the value of an environment variable based on the given key.

This function takes a key and returns the corresponding environment variable's value. It handles different cases based on the first character of the key:

- If the key starts with '$', it skips the '$' and looks up the environment variable.
- If the key starts with '"', it expands multiple variables within the key.
- If the key starts with '\', it trims the quotes from the key.
- Otherwise, it returns a duplicate of the key.

Additionally, if the key is "?", it returns the shell's exit code as a string.

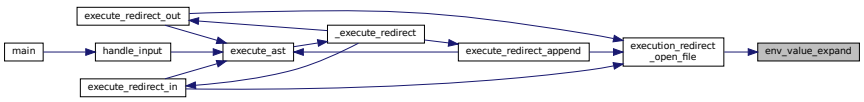
Parameters

shell	A pointer to the shell structure containing environment variables and exit code.
key	The key of the environment variable to expand.

Returns

A dynamically allocated string containing the expanded value, or NULL if the key is invalid or the value is not found.

Here is the caller graph for this function:



4.4.2.2 enviroment_node_create()

```
t_environment_node* enviroment_node_create (
            const char * entry,
            t_shell * shell,
            t_bool is_private )
```

Creates an environment node from an entry string and adds it to the shell's environment list.

This function uses environment_node_from_entry to parse the entry string into a node. If successful, it inserts this node into the shell's environment list using _environment_node_insert. It returns the created node, or NULL if parsing fails.

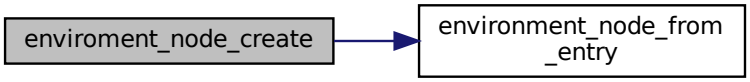
Parameters

entry	The entry string to process.
shell	A pointer to the shell context containing the environment list.

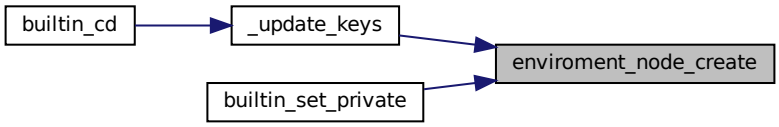
Returns

t_environment_node* A pointer to the newly created and inserted node, or NULL.

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.2.3 enviroment_node_delete()

```
void enviroment_node_delete (
            const char * key,
            t_shell * shell )
```

Deletes an environment node with a specified key from the shell's environment list.

This function searches for a node with the given key in the shell's environment list. If found, it removes the node from the list and frees its resources. If not found, or if inputs are invalid, it does nothing.

Parameters

key	The key of the node to delete.
shell	A pointer to the shell context containing the environment list.

Here is the call graph for this function:

Here is the caller graph for this function:

4.4.2.4 environment_list_clear()

```
void environment_list_clear (
            t_shell * shell )
```

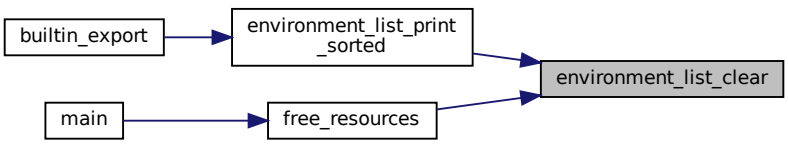
Clears all nodes in the shell's environment list.

This function iterates through each node in the environment list, deletes them one by one using _environment_↩ node_delete, and updates the head pointer to NULL. It ensures that all resources are properly freed and the list is completely cleared.

Parameters

shell	A pointer to the shell context containing the environment list.

Here is the caller graph for this function:



4.4.2.5 environment_list_get_sorted_copy()

```
t_environment_node* environment_list_get_sorted_copy (
            t_environment_node * original )
```

Creates a sorted copy of the environment list.

This function takes an unsorted linked list of environment variables and returns a new linked list that is sorted by the keys of the environment variables. The original list remains unchanged.

Parameters

original	A pointer to the head of the original unsorted environment list.

Returns

A pointer to the head of the new sorted environment list.

Here is the caller graph for this function:

4.4.2.6 environment_list_initialize()

```
t_environment_node* environment_list_initialize (
            const char ** envp )
```

Initializes an environment list from an array of environment variable strings.

This function processes each string in the provided array using environment_node_from_entry to create nodes. Each valid node is inserted into the environment list. It returns the head of the newly created list, which may be empty if all entries are invalid.

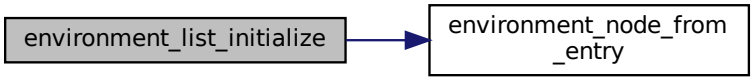
Parameters

envp	An array of strings representing environment variables.

Returns

t_environment_node* The head of the environment list, or NULL if no nodes were created.

Here is the call graph for this function:



Here is the caller graph for this function:

4.4.2.7 environment_list_print()

```
void environment_list_print (
            t_shell * shell,
            t_bool is_export )
```

Prints the environment variables in the shell.

This function iterates through the linked list of environment variables and prints each variable in the format specified by the `is_export` flag.

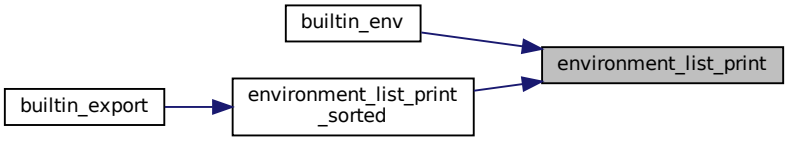
Parameters

shell	Pointer to the shell structure containing the environment list.
is_export	Boolean flag indicating the format of the output: • If true, prints variables in the format used by the `export` command. • If false, prints variables in the standard key=value format.

Returns

void

Here is the caller graph for this function:



4.4.2.8 environment_list_print_sorted()

```
void environment_list_print_sorted (
            t_shell * shell )
```

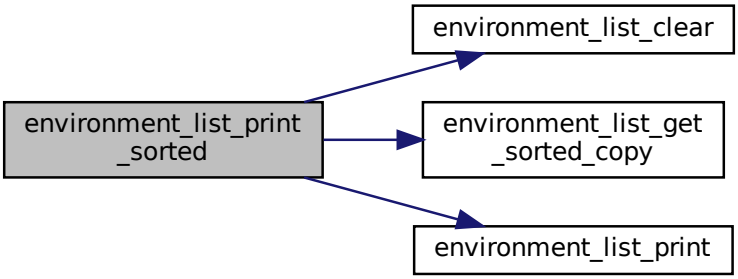
Prints the environment variables in sorted order.

This function creates a temporary copy of the environment variables, sorts them, and then prints them. After printing, it clears the temporary environment list to free up memory.

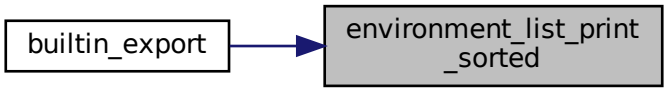
Parameters

shell	A pointer to the shell structure containing the environment list.

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.2.9 environment_list_read()

```
char* environment_list_read (
            const char * key,
            t_shell * shell )
```

Reads the value associated with a given key from the environment list.

This function traverses the linked list of environment variables stored in the shell structure and returns the value associated with the specified key.

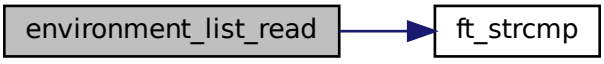
Parameters

key	The key to search for in the environment list.
shell	A pointer to the shell structure containing the environment list.

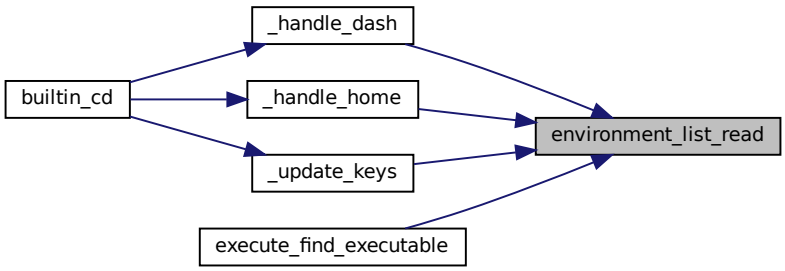
Returns

The value associated with the specified key, or an empty string if the key is not found.

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.2.10 environment_list_read_node()

```
t_environment_node* environment_list_read_node (
            const char * key,
            t_shell * shell )
```

Reads a node from the environment list based on the given key.

This function searches through the environment list in the shell structure to find a node that matches the provided key. If a matching node is found, it is returned. Otherwise, the function returns NULL.

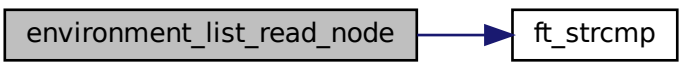
Parameters

key	The key to search for in the environment list.
shell	A pointer to the shell structure containing the environment list.

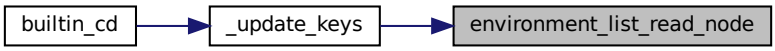
Returns

t_environment_node* A pointer to the matching environment node, or NULL if not found.

Here is the call graph for this function:



Here is the caller graph for this function:



4.4.2.11 environment_node_from_entry()

```
t_environment_node* environment_node_from_entry (
            const char * entry,
            t_bool is_private )
```

Creates a new environment node from an entry string.

This function takes an entry string, trims any surrounding double quotes, and splits it into key-value pairs using the '=' delimiter. It validates that exactly two parts exist after splitting. If invalid, it cleans up resources and returns NULL. Otherwise, it initializes a new node with the parsed values.

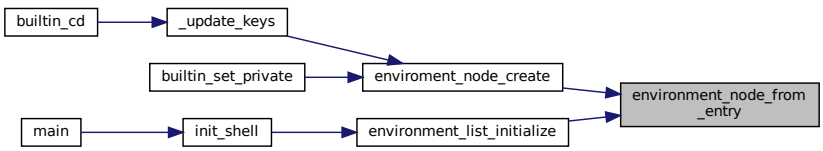
Parameters

entry	The entry string to parse.

Returns

t_environment_node* A pointer to the newly created node, or NULL if parsing failed.

Here is the caller graph for this function:



4.4.2.12 environment_serialize()

```
char** environment_serialize (
            t_shell * shell )
```

Serializes the environment variables from the shell into an array of strings.

This function takes the environment variables stored in the shell's environment list and serializes them into an array of strings, where each string is in the format "key=value". Private environment variables (marked by `is_private`) are excluded from the serialization.

Parameters

shell	A pointer to the shell structure containing the environment list.

Returns

A NULL-terminated array of strings representing the serialized environment variables. Returns NULL if there are no environment variables or if memory allocation fails.

4.4.2.13 environment_serialized_list_clear()

```
void environment_serialized_list_clear (
            char ** list )
```

Frees a list of strings and the list itself.

This function iterates through a list of strings, freeing each string, and then frees the list itself.

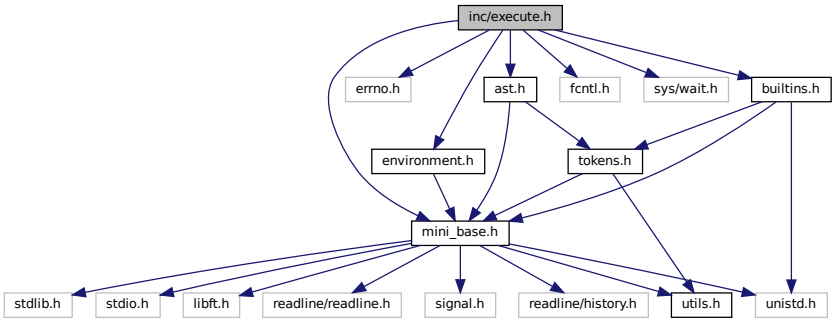
Parameters

list	A null-terminated array of strings to be freed.

4.5 inc/execute.h File Reference

```
#include <mini_base.h>
#include <errno.h>
#include <environment.h>
#include <builtins.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <ast.h>
```

Include dependency graph for execute.h:



This graph shows which files directly or indirectly include this file:

Classes

- struct s_heredoc_data

Macros

- #define HEREDOC_FILE_TEMPLATE "/tmp/minishell_heredoc_"

Typedefs

- typedef struct s_shell t_shell
- typedef struct s_heredoc_data t_heredoc_data

Functions

- char * execute_find_executable (char *command, t_shell *shell)

  Finds the full path of an executable command.
- void execute_command_node (t_shell *shell, t_ast_node *node)

  Executes a command node in the shell's abstract syntax tree (AST).
- void execute_ast (t_shell *shell, t_ast_node *node)

  Executes the abstract syntax tree (AST) node.
- void execute_redirect_in (t_shell *shell, t_ast_node *node)

  Executes a command with input redirection.
- void execute_pipe (t_shell *shell, t_ast_node *node)

  Executes a command pipeline by forking two child processes.
- void execute_redirect_append (t_shell *shell, t_ast_node *node)
- void execute_redirect_out (t_shell *shell, t_ast_node *node)

  Executes a command with output redirected to a file.
- void execute_preprocess_heredocs (t_ast_node *node)

  Preprocesses heredoc nodes in the abstract syntax tree (AST).
- int execution_redirect_open_file (t_shell *shell, t_ast_node *node)

  Opens a file for redirection during command execution.

4.5.1 Macro Definition Documentation

4.5.1.1 HEREDOC_FILE_TEMPLATE

```
#define HEREDOC_FILE_TEMPLATE "/tmp/minishell_heredoc_"
```

4.5.2 Typedef Documentation

4.5.2.1 t_heredoc_data

```
typedef struct s_heredoc_data t_heredoc_data
```

4.5.2.2 t_shell

```
typedef struct s_shell t_shell
```

4.5.3 Function Documentation

4.5.3.1 execute_ast()

```
void execute_ast (
            t_shell * shell,
            t_ast_node * node )
```

Executes the abstract syntax tree (AST) node.

This function takes a shell context and an AST node, and executes the node based on its type. The function handles different types of nodes such as command nodes, pipe nodes, and various types of redirection nodes.

Parameters

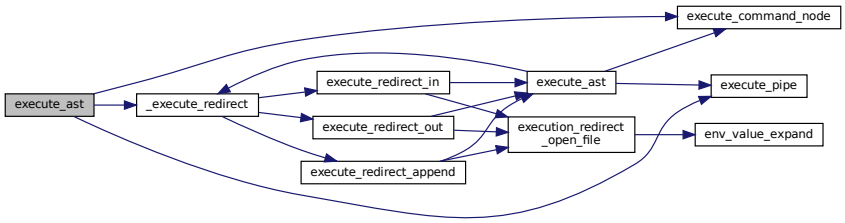
shell	A pointer to the shell context.
node	A pointer to the AST node to be executed.

Note

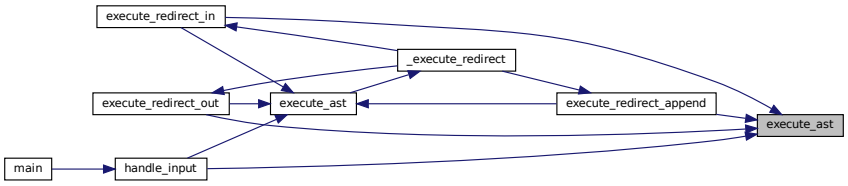
If the node is NULL, the function returns immediately.

The function delegates the execution to specific functions based on the type of the node.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.2 execute_command_node()

```
void execute_command_node (
            t_shell * shell,
            t_ast_node * node )
```

Executes a command node in the shell's abstract syntax tree (AST).

This function determines if the command node represents a built-in command or a more complex command. If it is a built-in command, it executes it directly. Otherwise, it forks a new process to execute the complex command.

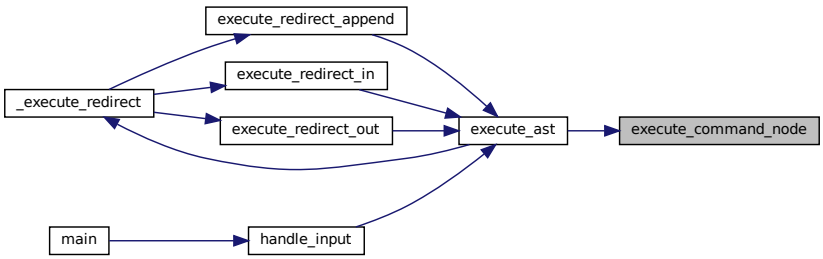
Parameters

shell	A pointer to the shell structure containing the shell's state.
node	A pointer to the AST node representing the command to be executed.

Note

If the command is not a built-in, the function forks a new process. The parent process waits for the child process to complete and updates the shell's exit code based on the child's exit status.

Here is the caller graph for this function:



4.5.3.3 execute_find_executable()

```
char* execute_find_executable (
            char * command,
            t_shell * shell )
```

Finds the full path of an executable command.

This function searches for the given command in the directories specified by the PATH environment variable. If the command is an absolute path or a relative path starting with "./", it checks if the command is executable.

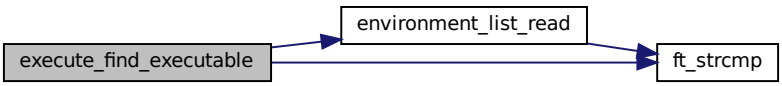
Parameters

command	The command to find the executable path for.
shell	The shell structure containing environment variables.

Returns

A dynamically allocated string containing the full path of the executable if found, or NULL if the command is not found or not executable.

Here is the call graph for this function:



4.5.3.4 execute_pipe()

```
void execute_pipe (
            t_shell * shell,
            t_ast_node * node )
```

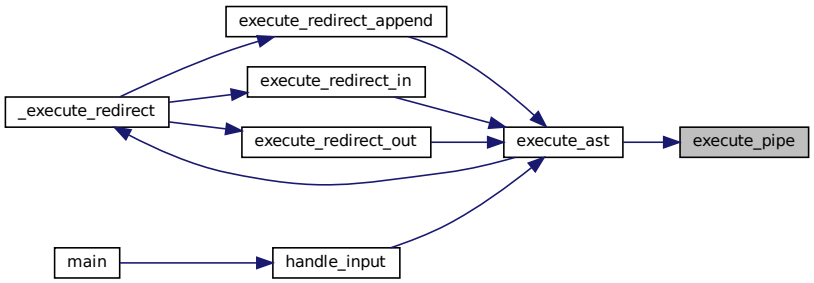
Executes a command pipeline by forking two child processes.

This function creates a pipe and forks two child processes to execute the left and right sides of a pipeline. The left child process writes to the pipe, and the right child process reads from the pipe. The parent process waits for both child processes to complete and sets the shell's exit code based on the status of the right child process.

Parameters

shell	A pointer to the shell structure.
node	A pointer to the abstract syntax tree node representing the pipeline.

Here is the caller graph for this function:



4.5.3.5 execute_preprocess_heredocs()

```
void execute_preprocess_heredocs (
            t_ast_node * node )
```

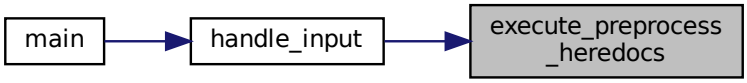
Preprocesses heredoc nodes in the abstract syntax tree (AST).

This function recursively traverses the AST and processes nodes of type TOKEN_HEREDOC. For each such node, it trims the quotes from the delimiter, creates a temporary file name, processes the heredoc input, and updates the node type to TOKEN_REDIRECT_IN. It also updates the node's arguments to point to the temporary file name.

Parameters

node	A pointer to the current AST node being processed.

Here is the caller graph for this function:



4.5.3.6 execute_redirect_append()

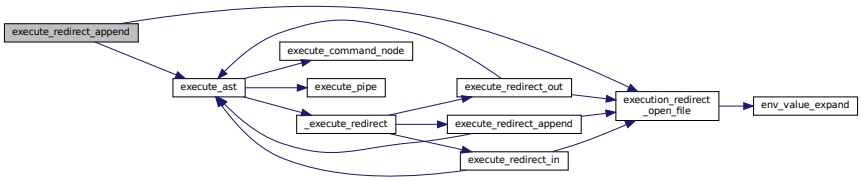
```
void execute_redirect_append (
            t_shell * shell,
            t_ast_node * node )
```

execute_redirect_append - Executes a command with output redirected to a file in append mode. @shell: Pointer to the shell structure containing environment variables and state. @node: Pointer to the AST node representing the command and redirection.

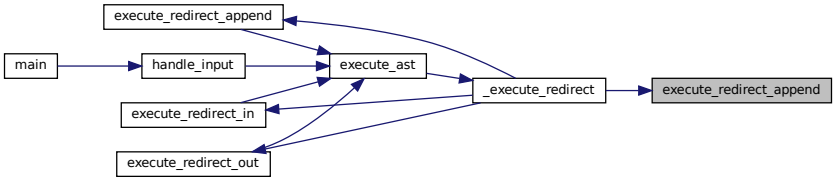
This function performs the following steps:

1. Duplicates the current STDOUT file descriptor to save the original STDOUT.
2. Expands the file name using environment variable expansion.
3. Opens the file in append mode, creating it if it does not exist.
4. Redirects STDOUT to the opened file.
5. Executes the command represented by the left child of the AST node.
6. Restores the original STDOUT.
7. Closes the file descriptors.

If the file cannot be opened, the function prints an error message and exits. Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.7 execute_redirect_in()

```
void execute_redirect_in (
            t_shell * shell,
            t_ast_node * node )
```

Executes a command with input redirection.

This function handles the execution of a command where the input is redirected from a file. It temporarily replaces the standard input (stdin) with the contents of the specified file, executes the command, and then restores the original stdin.

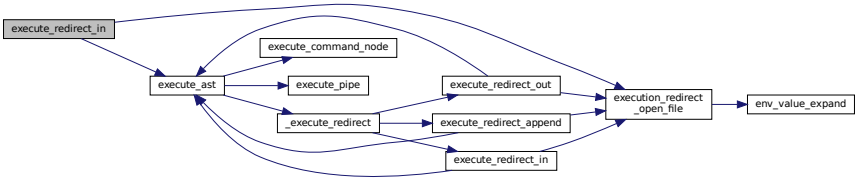
Parameters

shell	Pointer to the shell structure containing environment and state information.
node	Pointer to the AST node representing the command and its redirection.

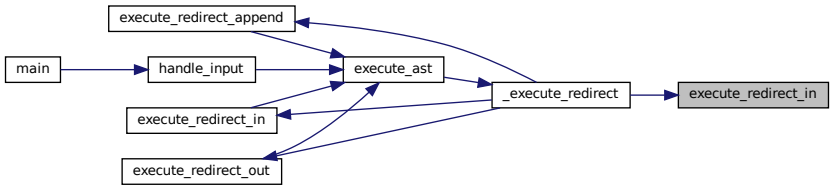
The function performs the following steps:

1. Duplicates the current stdin file descriptor to save it.
2. Expands the file name using environment variable expansion.
3. Opens the file for reading.
4. If the file cannot be opened, prints an error message and exits.
5. Redirects stdin to the opened file.
6. Executes the command represented by the left child of the AST node.
7. Restores the original stdin.
8. Closes the duplicated stdin file descriptor.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.8 execute_redirect_out()

```
void execute_redirect_out (
            t_shell * shell,
            t_ast_node * node )
```

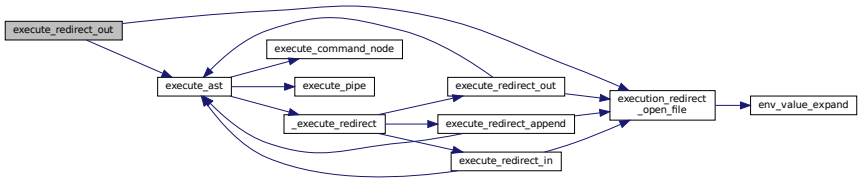
Executes a command with output redirected to a file.

This function handles the redirection of the standard output (stdout) to a file specified in the abstract syntax tree (AST) node. It first duplicates the current stdout file descriptor to restore it later. Then, it expands the file name using environment variable expansion, opens the file with write permissions, and redirects stdout to this file. After executing the command represented by the left child of the AST node, it restores the original stdout file descriptor.

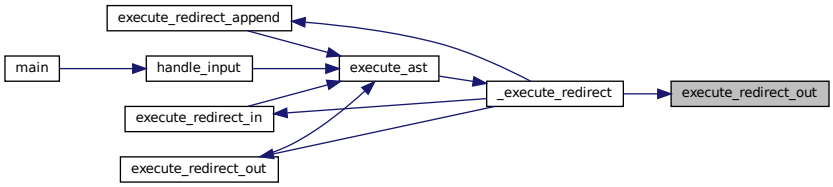
Parameters

shell	Pointer to the shell structure containing the environment and state.
node	Pointer to the AST node representing the redirection operation.

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.3.9 execution_redirect_open_file()

```
int execution_redirect_open_file (
            t_shell * shell,
            t_ast_node * node )
```

Opens a file for redirection during command execution.

This function expands the file name using environment variables, opens the file with the appropriate mode, and returns the file descriptor. If the file cannot be opened, the function will terminate the program with an exit status of failure.

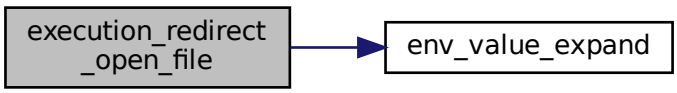
Parameters

shell	A pointer to the shell structure containing environment variables.
node	A pointer to the AST node containing the file redirection information.

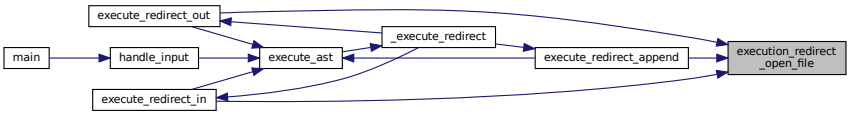
Returns

The file descriptor of the opened file.

Here is the call graph for this function:



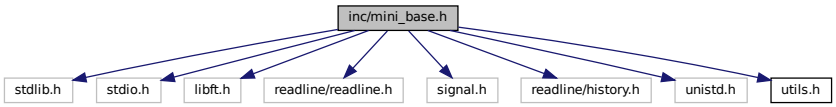
Here is the caller graph for this function:



4.6 inc/mini_base.h File Reference

```
#include <stdlib.h>
#include <stdio.h>
#include <libft.h>
#include <readline/readline.h>
#include <signal.h>
#include <readline/history.h>
#include <unistd.h>
#include <utils.h>
```

Include dependency graph for mini_base.h:



This graph shows which files directly or indirectly include this file:

Typedefs

- typedef enum s_bool t_bool

Enumerations

- enum s_bool { false , true }

Functions

- void signals_setup (void)

  Sets up signal handlers for the application.

4.6.1 Typedef Documentation

4.6.1.1 t_bool

```
typedef enum s_bool t_bool
```

4.6.2 Enumeration Type Documentation

4.6.2.1 s_bool

```
enum s_bool
```

Enumerator

false	
true	

4.6.3 Function Documentation

4.6.3.1 signals_setup()

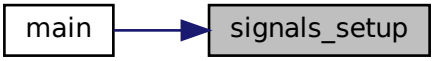
```
void signals_setup (
            void  )
```

Sets up signal handlers for the application.

This function configures the signal handlers for SIGINT and SIGQUIT signals.

- SIGINT: When the SIGINT signal is received (typically generated by pressing Ctrl+C), the `signals_handle_sigint` function will be called to handle the signal.
- SIGQUIT: The SIGQUIT signal (typically generated by pressing Ctrl+) will be ignored.

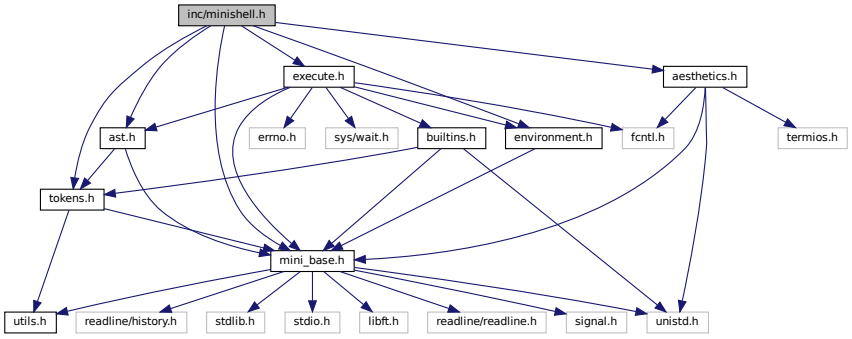
Here is the caller graph for this function:



4.7 inc/minishell.h File Reference

```
#include <mini_base.h>
#include <environment.h>
#include <tokens.h>
#include <ast.h>
#include <execute.h>
#include <aesthetics.h>
```

Include dependency graph for minishell.h:



This graph shows which files directly or indirectly include this file:

Classes

- struct s_shell

Macros

- #define PROMPT "\033[35m minishell > \033[0m"

Typedefs

- typedef struct s_shell t_shell

4.7.1 Macro Definition Documentation

4.7.1.1 PROMPT

```
#define PROMPT "\033[35m minishell > \033[0m"
```

4.7.2 Typedef Documentation

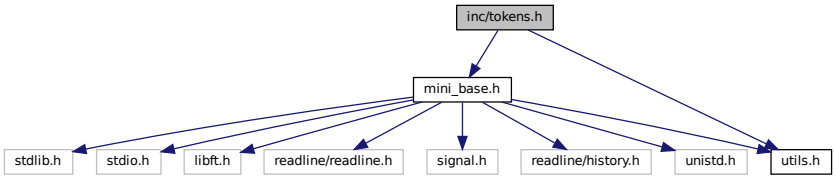
4.7.2.1 t_shell

```
typedef struct s_shell t_shell
```

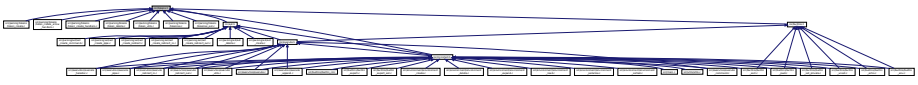
4.8 inc/tokens.h File Reference

```
#include <mini_base.h>
#include <utils.h>
```

Include dependency graph for tokens.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct s_token_node

Typedefs

- typedef enum s_token_type t_token_type
- typedef struct s_token_node t_token_node

Enumerations

- enum s_token_type {
  TOKEN_STRING , TOKEN_PIPE , TOKEN_REDIRECT_IN , TOKEN_REDIRECT_OUT ,
  TOKEN_APPEND , TOKEN_HEREDOC }

Functions

- void token_delete_all (t_token_node **list)

  Frees the memory allocated for a list of tokens.
- void token_delete (t_token_node *token)

  Frees the memory allocated for a token and its arguments.
- t_token_node * token_create (t_token_type type, char *value)

  Creates a new token node with the specified type and value.
- void token_print (t_token_node *token)

  Prints the details of a single token node and its arguments.
- t_token_node ** tokenise (char *input)
- void handle_simple_tokens (char **current, t_token_node **list)

  Handles simple tokens in the input string.
- void handle_arrows (t_token_node **list, char **current)

  Handles the arrows in the input string.
- void handle_quoted (char **current, t_token_node **list, char type)

  Handles quoted tokens.
- void handle_word (t_token_node **list, char **current)

  Handles a word in the tokenization process.
- void token_list_print (t_token_node **head)

  Prints the details of a list of token nodes.
- void token_append (t_token_node **list, t_token_node *new_node)

  Appends a new token node to the end of the token list.
- size_t token_count_args (t_token_node *src)

  Counts the number of consecutive tokens of the same type as the source token.

4.8.1 Typedef Documentation

4.8.1.1 t_token_node

```
typedef struct s_token_node t_token_node
```

4.8.1.2 t_token_type

```
typedef enum s_token_type t_token_type
```

4.8.2 Enumeration Type Documentation

4.8.2.1 s_token_type

```
enum s_token_type
```

Enumerator

TOKEN_STRING	
TOKEN_PIPE	
TOKEN_REDIRECT_IN	
TOKEN_REDIRECT_OUT	
TOKEN_APPEND	
TOKEN_HEREDOC	

4.8.3 Function Documentation

4.8.3.1 handle_arrows()

```
void handle_arrows (
            t_token_node ** list,
            char ** current )
```

Handles the arrows in the input string.

This function determines whether the current character is '<' or '>'. If it is '<', it calls the handle_arrows_left() function. If it is '>', it calls the handle_arrows_right() function.

Parameters

current	The current character in the input string.
list	The token list to be updated.

Here is the caller graph for this function:

4.8.3.2 handle_quoted()

```
void handle_quoted (
            char ** current,
            t_token_node ** list,
            char type )
```

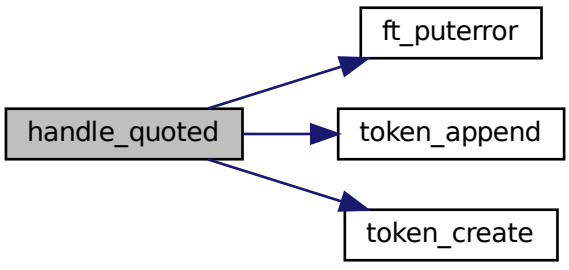
Handles quoted tokens.

This function is responsible for handling quoted tokens in the tokenisation process. It takes a pointer to the current character, a token list, and the type of quote. It searches for the closing quote and creates a token with the quoted content. If the closing quote is missing, it prints an error message.

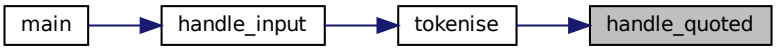
Parameters

current	A pointer to the current character.
list	The token list to add the created token to.
type	The type of quote.

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.3.3 handle_simple_tokens()

```
void handle_simple_tokens (
            char ** current,
            t_token_node ** list )
```

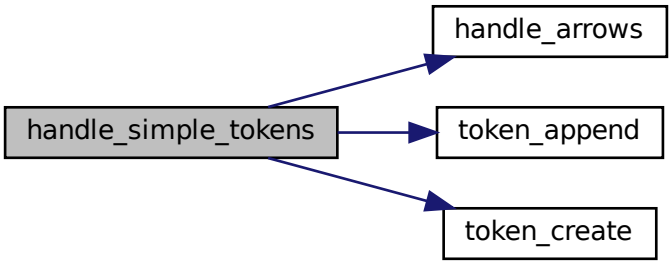
Handles simple tokens in the input string.

This function takes a pointer to the current character in the input string and a pointer to a token list. It checks the current character and adds the corresponding token to the token list. The current character pointer is updated accordingly.

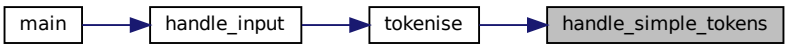
Parameters

current	Pointer to the current character in the input string.
list	Pointer to the token list.

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.3.4 handle_word()

```
void handle_word (
            t_token_node ** list,
            char ** current )
```

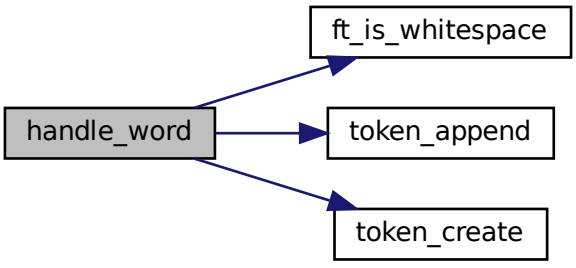
Handles a word in the tokenization process.

This function takes a pointer to the current character and a token list. It iterates through the characters starting from the current position until it reaches the end of the string or encounters a whitespace character, a special character ('&', '|', '(', ')', '<', '>'), or a null character ('\0'). It then adds a token of type TOKEN_WORD to the token list, containing the substring from the start position to the current position.

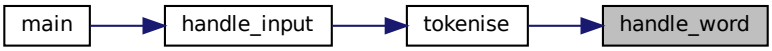
Parameters

current	A pointer to the current character.
list	The token list to add the token to.

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.3.5 token_append()

```
void token_append (
            t_token_node ** list,
            t_token_node * new_node )
```

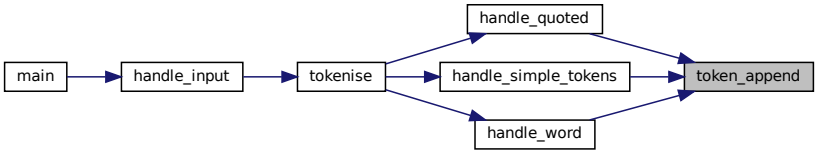
Appends a new token node to the end of the token list.

This function takes a pointer to the head of a token list and a new token node, and appends the new node to the end of the list. If the new node is NULL, the function does nothing. If the list is empty, the new node becomes the head of the list.

Parameters

list	A double pointer to the head of the token list.
new_node	A pointer to the new token node to be appended.

Here is the caller graph for this function:



4.8.3.6 token_count_args()

```
size_t token_count_args (
            t_token_node * src )
```

Counts the number of consecutive tokens of the same type as the source token.

This function iterates through the linked list of tokens starting from the next token of the given source token (`src`). It counts how many consecutive tokens have the same type as the source token and are of type `TOKEN_STRING`.

Parameters

src	A pointer to the source token node from which to start counting.

Returns

The number of consecutive tokens of the same type as the source token.

4.8.3.7 token_create()

```
t_token_node* token_create (
            t_token_type type,
            char * value )
```

Creates a new token node with the specified type and value.

This function initializes a new token node using the provided type and value. If the initialization fails, it returns NULL.

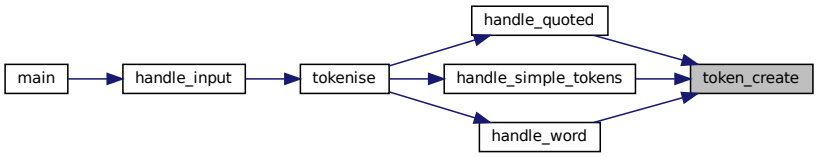
Parameters

type	The type of the token.
value	The value of the token.

Returns

A pointer to the newly created token node, or NULL if initialization fails.

Here is the caller graph for this function:



4.8.3.8 token_delete()

```
void token_delete (
            t_token_node * token )
```

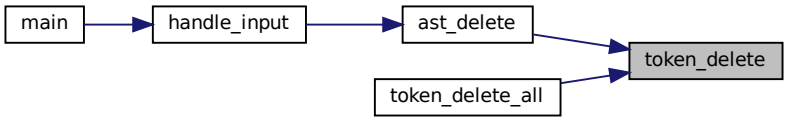
Frees the memory allocated for a token and its arguments.

This function releases the memory allocated for the arguments of a token and then frees the memory allocated for the token itself.

Parameters

token	A pointer to the token to be deleted.

Here is the caller graph for this function:



4.8.3.9 token_delete_all()

```
void token_delete_all (
            t_token_node ** list )
```

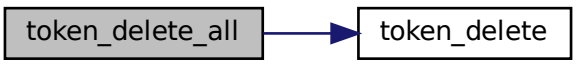
Frees the memory allocated for a list of tokens.

This function iterates through a linked list of tokens, freeing the memory allocated for each token and its arguments. It then frees the memory allocated for the list itself.

Parameters

list	A double pointer to the head of the list of tokens to be deleted.

Here is the call graph for this function:



4.8.3.10 token_list_print()

```
void token_list_print (
            t_token_node ** head )
```

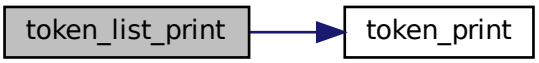
Prints the details of a list of token nodes.

This function checks if the head of the token node list is valid and then calls `token_print` to print the details of the token nodes.

Parameters

head	A double pointer to the head of the token node list.

Here is the call graph for this function:



4.8.3.11 token_print()

```
void token_print (
            t_token_node * token )
```

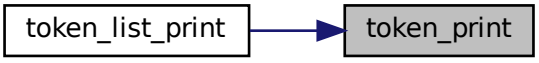
Prints the details of a single token node and its arguments.

This function iterates through a linked list of token nodes and prints the type, command/file, and arguments of each token node. It uses the helper function `_token_type_to_string` to convert the token type to a string for printing.

Parameters

token	A pointer to the first token node in the linked list.

Here is the caller graph for this function:



4.8.3.12 tokenise()

```
t_token_node** tokenise (
            char * input )
```

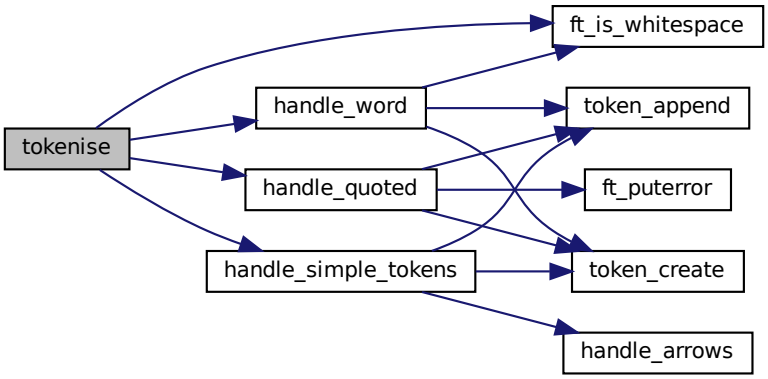
tokenise - Tokenizes the input string into a list of tokens. @input: The input string to be tokenized.

This function takes an input string and tokenizes it into a list of tokens. It handles different types of tokens including simple tokens (like '(', ')', '|', '<', '>'), quoted strings, and words. The function skips over whitespace and processes each token accordingly. The resulting list of tokens is returned.

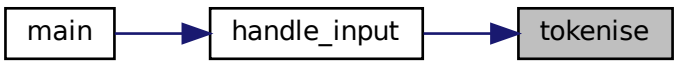
Returns

: A pointer to the list of tokens, or NULL if memory allocation fails.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9 inc/utils.h File Reference

This graph shows which files directly or indirectly include this file:

Typedefs

- typedef enum s_bool t_bool
- typedef struct s_shell t_shell

Functions

- int ft_strcmp (const char *s1, const char *s2)

  Compares two null-terminated strings lexicographically.
- t_bool ft_is_whitespace (char c)

  Checks if a character is a whitespace character.
- void ft_puterror (const char *s, char *arg1, const char *arg2)

  Writes an error message to the standard error output.
- void clear_screen_ensure_cursor_visible (void)

  Clears the terminal screen and ensures the cursor is visible at the top-left corner.
- void free_resources (t_shell *shell)

  Frees resources associated with the shell.
- void init_shell (t_shell *shell, const char **envp)

  Initializes the shell structure with environment variables and default settings.
- void handle_input (t_shell *shell)

  Handles the input loop for the shell.

4.9.1 Typedef Documentation

4.9.1.1 t_bool

```
typedef enum s_bool t_bool
```

4.9.1.2 t_shell

```
typedef struct s_shell t_shell
```

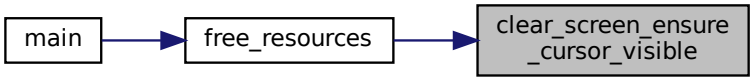
4.9.2 Function Documentation

4.9.2.1 clear_screen_ensure_cursor_visible()

```
void clear_screen_ensure_cursor_visible (
            void  )
```

Clears the terminal screen and ensures the cursor is visible at the top-left corner.

This function sends ANSI escape codes to the terminal to clear the screen and move the cursor to the home position (top-left corner). It is useful for refreshing the terminal display. Here is the caller graph for this function:



4.9.2.2 free_resources()

```
void free_resources (
            t_shell * shell )
```

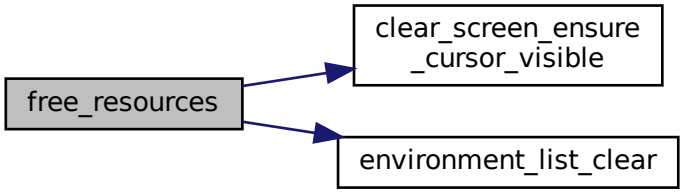
Frees resources associated with the shell.

This function clears the environment list and ensures the screen is cleared and the cursor is visible.

Parameters

shell	A pointer to the shell structure containing resources to be freed.

Here is the call graph for this function:



Here is the caller graph for this function:

4.9.2.3 ft_is_whitespace()

```
t_bool ft_is_whitespace (
            char c )
```

Checks if a character is a whitespace character.

This function checks if the given character c is a whitespace character. Whitespace characters include space (' '), horizontal tab ('\t'), newline ('
'), and carriage return ('\r').

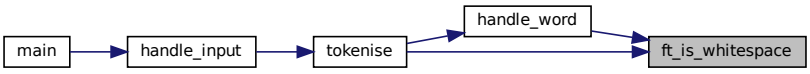
Parameters

c	The character to be checked.

Returns

A boolean value indicating whether the character is a whitespace character.

Here is the caller graph for this function:



4.9.2.4 ft_puterror()

```
void ft_puterror (
            const char * s,
```

```
            char * arg1,
            const char * arg2 )
```

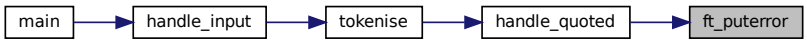
Writes an error message to the standard error output.

This function writes the provided error message and arguments to the standard error output (stderr). If the second argument is NULL, the function returns without writing anything. If the third argument is provided, it will also be written to stderr. A newline character is written at the end of the message.

Parameters

s	The main error message to be written. Must not be NULL.
arg1	The first argument to be written. Must not be NULL.
arg2	The second argument to be written. Can be NULL.

Here is the caller graph for this function:



4.9.2.5 ft_strcmp()

```
int ft_strcmp (
            const char * s1,
            const char * s2 )
```

Compares two null-terminated strings lexicographically.

This function compares the two strings s1 and s2. It returns an integer less than, equal to, or greater than zero if s1 is found, respectively, to be less than, to match, or be greater than s2.

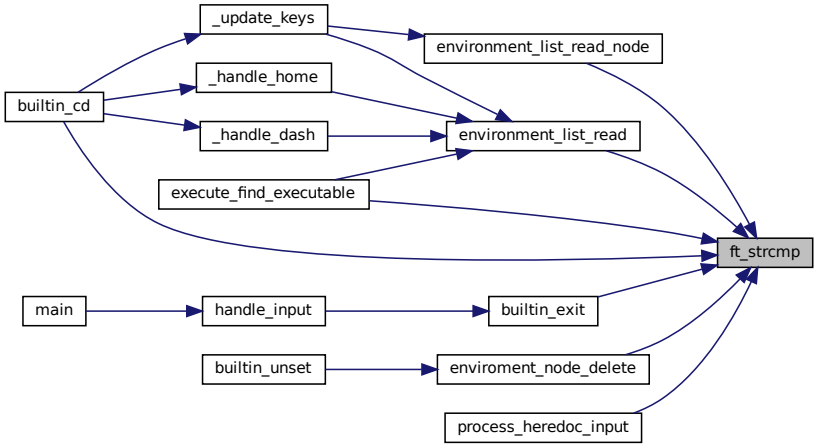
Parameters

s1	The first string to be compared.
s2	The second string to be compared.

Returns

An integer less than, equal to, or greater than zero if s1 is found, respectively, to be less than, to match, or be greater than s2.

Here is the caller graph for this function:



4.9.2.6 handle_input()

```
void handle_input (
            t_shell * shell )
```

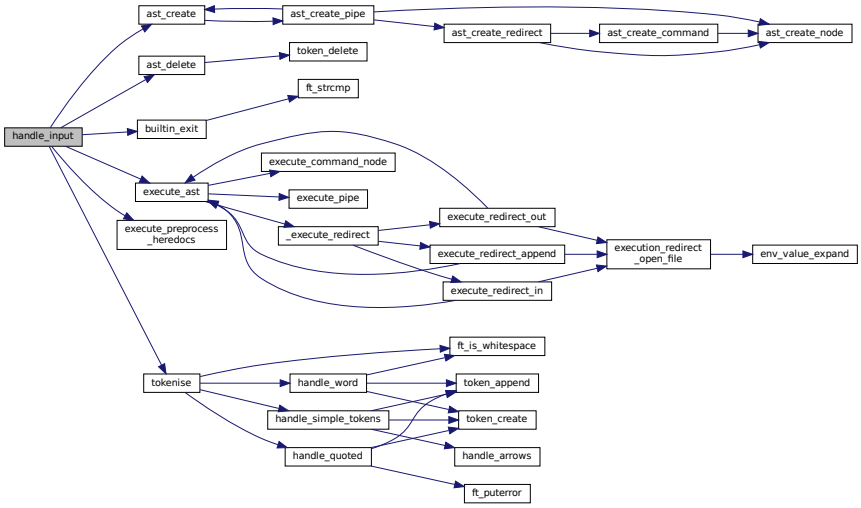
Handles the input loop for the shell.

This function continuously prompts the user for input, tokenizes the input, creates an abstract syntax tree (AST) from the tokens, preprocesses any heredocs, checks for the exit builtin command, executes the AST, and then cleans up the resources.

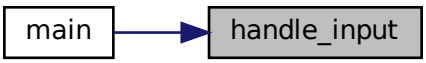
Parameters

shell	A pointer to the shell structure.

Here is the call graph for this function:



Here is the caller graph for this function:



4.9.2.7 init_shell()

```
void init_shell (
            t_shell * shell,
            const char ** envp )
```

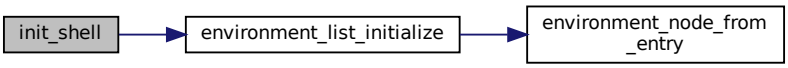
Initializes the shell structure with environment variables and default settings.

This function sets up the initial state of the shell by initializing the environment list, setting the exit code to 0, and disabling top-level redirections. If the environment list initialization fails, the function prints an error message and exits the program.

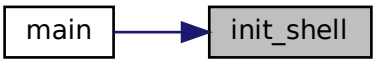
Parameters

shell	Pointer to the shell structure to be initialized.
envp	Array of environment variables.

Here is the call graph for this function:

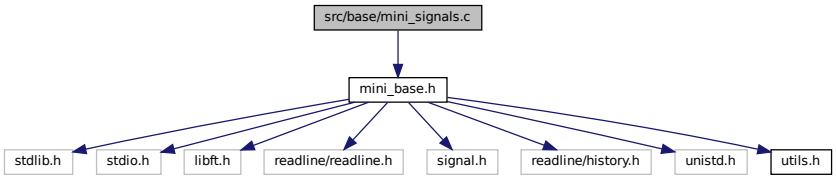


Here is the caller graph for this function:



4.10 src/base/mini_signals.c File Reference

`#include <mini_base.h>`
Include dependency graph for mini_signals.c:



Functions

- void signals_setup (void)

  Sets up signal handlers for the application.

4.10.1 Function Documentation

4.10.1.1 signals_setup()

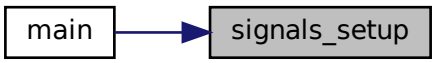
```
void signals_setup (
            void  )
```

Sets up signal handlers for the application.

This function configures the signal handlers for SIGINT and SIGQUIT signals.

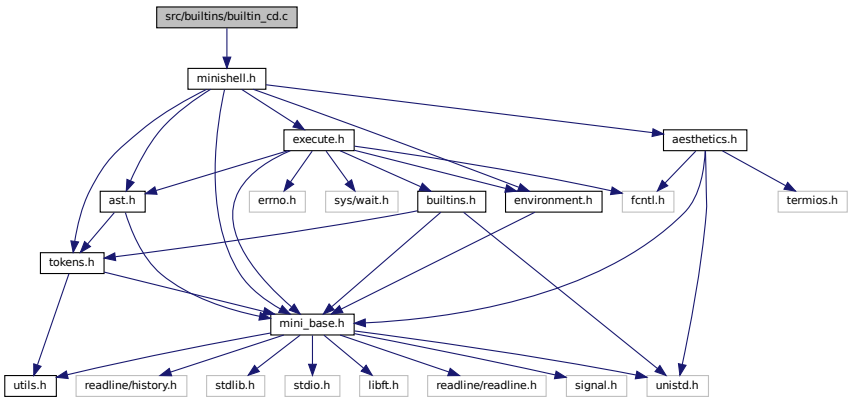
- SIGINT: When the SIGINT signal is received (typically generated by pressing Ctrl+C), the `signals_↩handle_sigint` function will be called to handle the signal.
- SIGQUIT: The SIGQUIT signal (typically generated by pressing Ctrl+) will be ignored.

Here is the caller graph for this function:



4.11 src/builtins/builtin_cd.c File Reference

`#include <minishell.h>`
Include dependency graph for builtin_cd.c:



Functions

- void _update_keys (t_shell ∗shell, char ∗curr_path)

  Updates the PWD and OLDPWD environment variables after a directory change.
- char ∗ _handle_dash (t_shell ∗shell, char ∗target)

  Handles the `cd -` case, switching to the previous directory.
- char ∗ _handle_home (t_shell ∗shell, char ∗target)

  Handles the `cd` command without arguments or with ∼, switching to the home directory.
- void builtin_cd (t_shell ∗shell, char ∗∗args)

  Changes the current working directory of the shell.

4.11.1 Function Documentation

4.11.1.1 _handle_dash()

```
char* _handle_dash (
            t_shell * shell,
            char * target )
```

Handles the `cd -` case, switching to the previous directory.

This function retrieves the value of `OLDPWD` and returns it. If `OLDPWD` is not set, an error message is printed.

Parameters

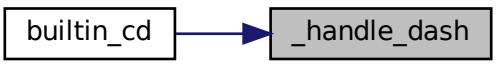
shell	The shell structure containing environment variables.
target	Unused in this function, only for consistency.

Returns

The path stored in `OLDPWD`, or NULL if not set.

Here is the call graph for this function:

Here is the caller graph for this function:



4.11.1.2 _handle_home()

```
char* _handle_home (
            t_shell * shell,
            char * target )
```

Handles the `cd` command without arguments or with ~, switching to the home directory.

This function retrieves the value of `HOME` and returns it. If `HOME` is not set, an error message is printed.

Parameters

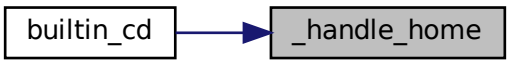
shell	The shell structure containing environment variables.
target	Unused in this function, only for consistency.

Returns

The path stored in `HOME`, or NULL if not set.

Here is the call graph for this function:

Here is the caller graph for this function:



4.11.1.3 _update_keys()

```
void _update_keys (
            t_shell * shell,
            char * curr_path )
```

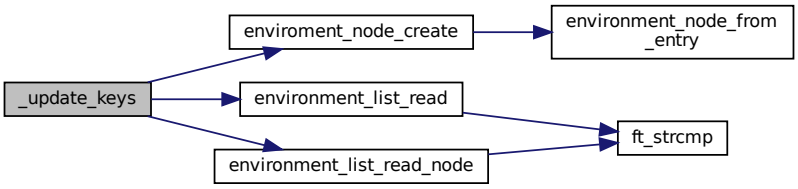
Updates the PWD and OLDPWD environment variables after a directory change.

This function modifies the shell's environment list to reflect the change in the working directory. It updates `OLDPWD` with the previous `PWD` value and sets `PWD` to the new current working directory.

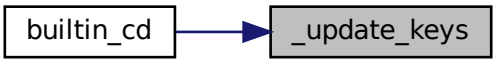
Parameters

shell	The shell structure containing environment variables.
curr_path	The new current working directory path.

Here is the call graph for this function:



Here is the caller graph for this function:



4.11.1.4 builtin_cd()

```
void builtin_cd (
            t_shell * shell,
            char ** args )
```

Changes the current working directory of the shell.

This function implements the `cd` built-in command, allowing users to navigate the filesystem. It supports:

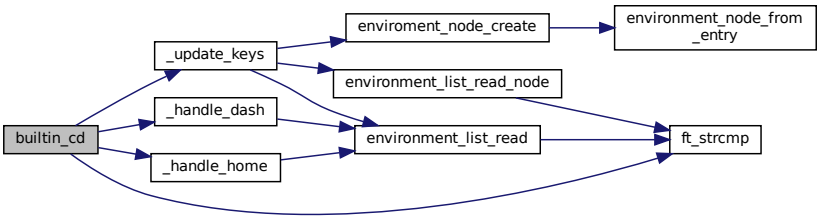
- `cd` or `cd ~` to move to the home directory.
- `cd -` to move to the previous working directory.
- `cd <directory>` to move to the specified directory.

If the directory change is successful, it updates the `PWD` and `OLDPWD` environment variables. If it fails, an error message is printed.

Parameters

shell	A pointer to the shell structure.
args	An array of arguments passed to the `cd` command.

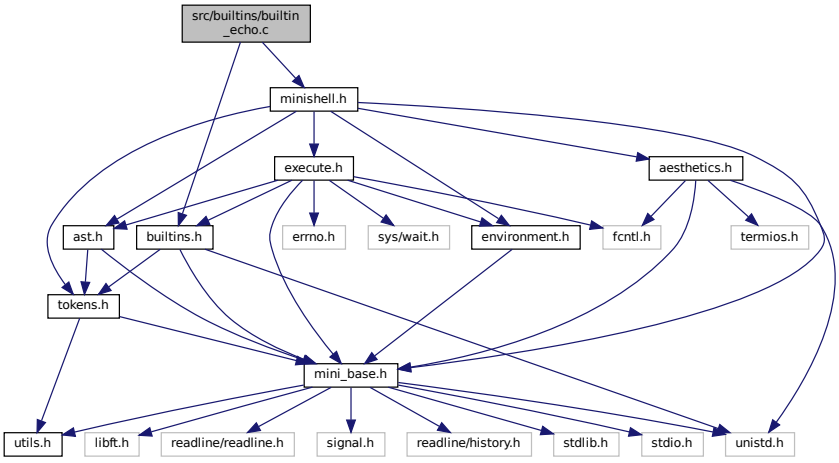
Here is the call graph for this function:



4.12 src/builtins/builtin_echo.c File Reference

```
#include <builtins.h>
#include <minishell.h>
```

Include dependency graph for builtin_echo.c:



Functions

- void builtin_echo (t_shell *shell, t_ast_node *node)

  Implements the `echo` built-in command for the minishell.

4.12.1 Function Documentation

4.12.1.1 builtin_echo()

```
void builtin_echo (
            t_shell * shell,
            t_ast_node * node )
```

Implements the `echo` built-in command for the minishell.

This function mimics the behavior of the UNIX `echo` command, supporting the `-n` option to suppress the trailing newline. It expands environment variables within arguments and prints the result to standard output.

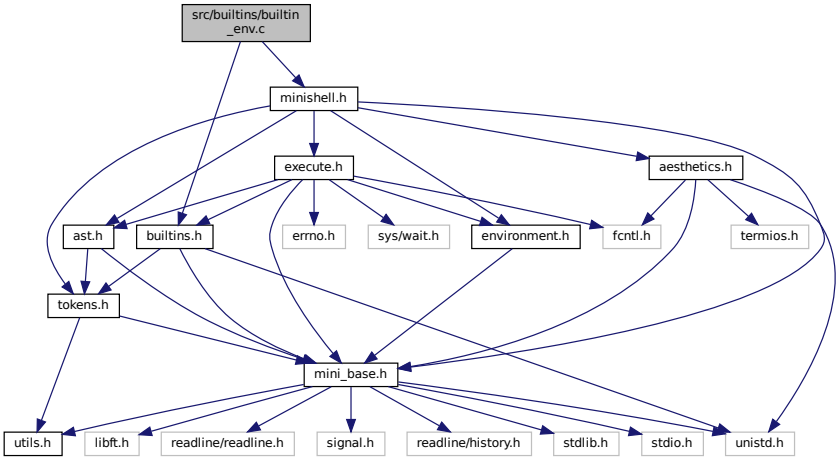
Parameters

shell	The shell structure containing environment variables and state.
node	The AST node representing the `echo` command and its arguments.

4.13 src/builtins/builtin_env.c File Reference

```
#include <builtins.h>
#include <minishell.h>
```

Include dependency graph for builtin_env.c:



Functions

- void builtin_env (t_shell *shell)

  Prints the current environment variables.

4.13.1 Function Documentation

4.13.1.1 builtin_env()

```
void builtin_env (
            t_shell * shell )
```

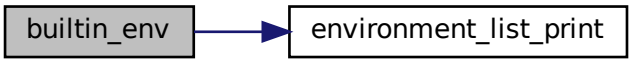
Prints the current environment variables.

This function prints all the environment variables stored in the shell's environment list. It also sets the shell's exit code to 0 upon completion.

Parameters

shell	A pointer to the shell structure containing the environment list.

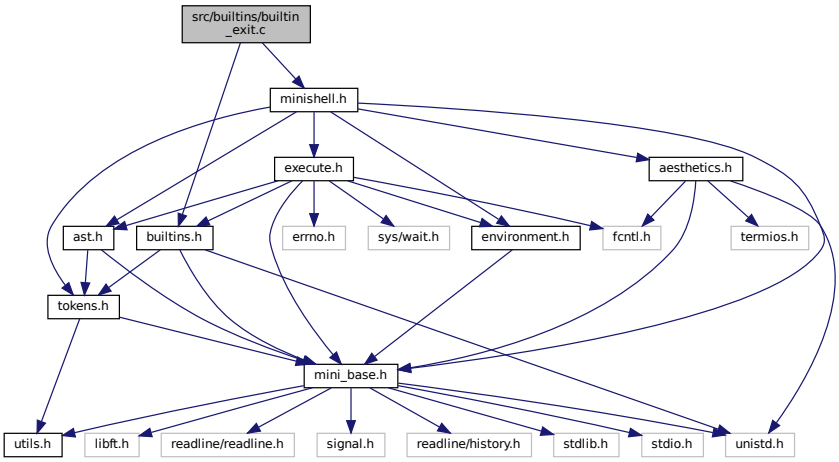
Here is the call graph for this function:



4.14 src/builtins/builtin_exit.c File Reference

```
#include <builtins.h>
#include <minishell.h>
```

Include dependency graph for builtin_exit.c:



Functions

- t_bool builtin_exit (t_shell *shell, t_ast_node **node_ptr, char **line_ptr, t_token_node ***list_ptr)

  Handles the execution of the `exit` built-in command.

4.14.1 Function Documentation

4.14.1.1 builtin_exit()

```
t_bool builtin_exit (
            t_shell * shell,
            t_ast_node ** node_ptr,
            char ** line_ptr,
            t_token_node *** list_ptr )
```

Handles the execution of the `exit` built-in command.

This function checks if the `exit` command is properly formatted and executes it, freeing necessary resources before terminating the shell. The function ensures:

- The command is correctly formatted with a single `exit` keyword.
- Argument validity is checked, allowing a numeric exit code.
- The shell's exit code is set appropriately.
- Memory cleanup is performed before exiting.

Parameters

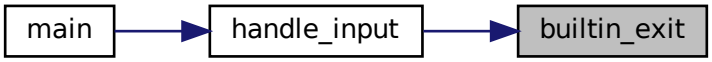
shell	A pointer to the shell structure.
node_ptr	A double pointer to the AST node containing the command.
line_ptr	A pointer to the command line input to be freed.
list_ptr	A pointer to the token list to be freed.

Returns

true if the `exit` command was executed, false otherwise.

Here is the call graph for this function:

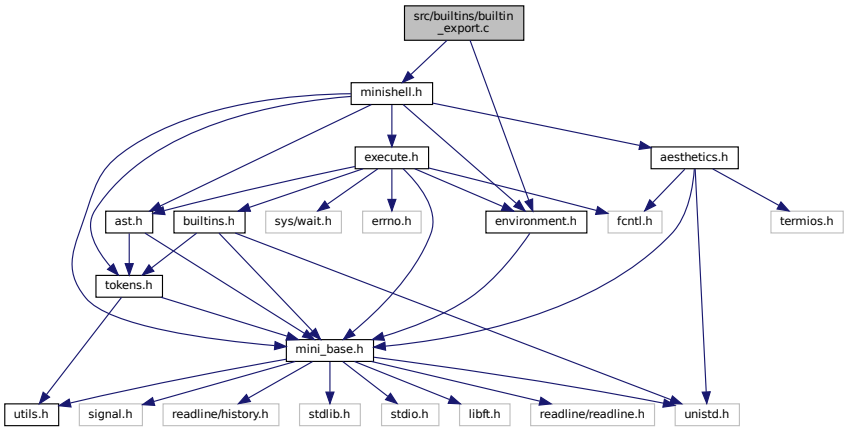
Here is the caller graph for this function:



4.15 src/builtins/builtin_export.c File Reference

```
#include <environment.h>
#include <minishell.h>
```

Include dependency graph for builtin_export.c:



Functions

- void builtin_export (t_shell *shell, t_ast_node *node)

  Handles the export builtin command in the shell.

4.15.1 Function Documentation

4.15.1.1 builtin_export()

```
void builtin_export (
            t_shell * shell,
            t_ast_node * node )
```

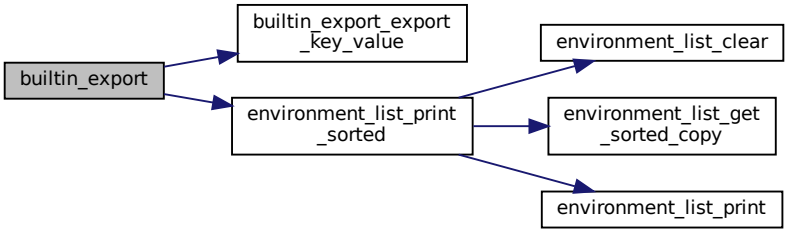
Handles the export builtin command in the shell.

This function processes the export command, which is used to set environment variables. If no arguments are provided, it prints the environment variables in sorted order. If arguments are provided, it attempts to set the specified environment variables.

Parameters

shell	A pointer to the shell structure containing the environment.
node	A pointer to the AST node representing the export command and its arguments.

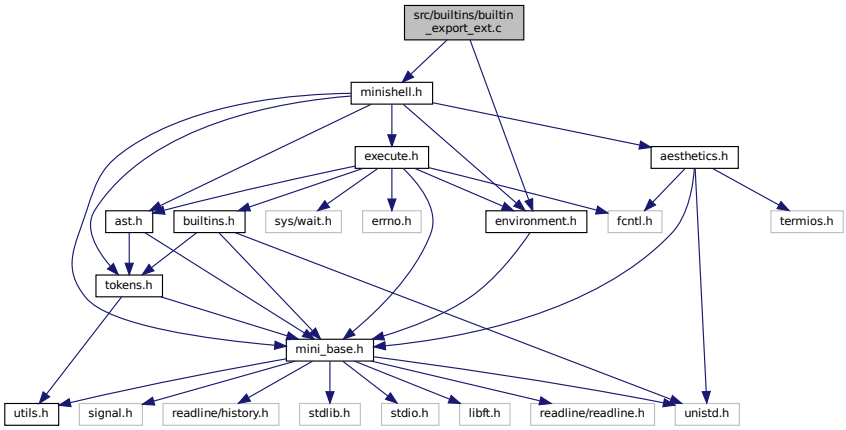
Here is the call graph for this function:



4.16 src/builtins/builtin_export_ext.c File Reference

```
#include <environment.h>
#include <minishell.h>
```

Include dependency graph for builtin_export_ext.c:



Functions

- t_bool builtin_export_export_key_value (t_shell *shell, t_ast_node *node)

  Processes the export command by handling multiple key-value pairs.

4.16.1 Function Documentation

4.16.1.1 builtin_export_export_key_value()

```
t_bool builtin_export_export_key_value (
            t_shell * shell,
            t_ast_node * node )
```

Processes the export command by handling multiple key-value pairs.

This function iterates through the provided arguments, processing each as either a single key or a key-value pair. It updates the environment variables accordingly.

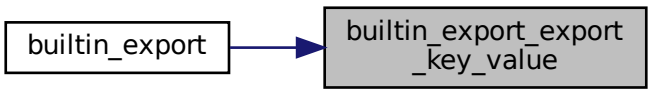
Parameters

shell	A pointer to the shell structure.
node	The AST node containing the command and its arguments.

Returns

true if an error occurs during processing, false otherwise.

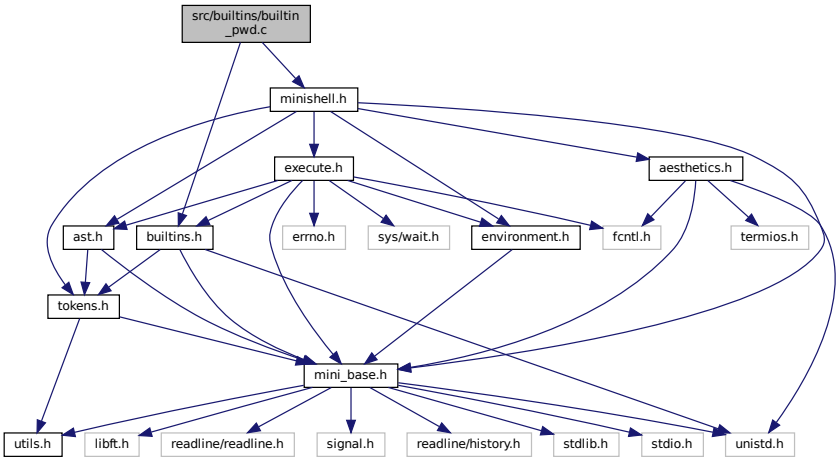
Here is the caller graph for this function:



4.17 src/builtins/builtin_pwd.c File Reference

```
#include <builtins.h>
#include <minishell.h>
```

Include dependency graph for builtin_pwd.c:



Functions

- char * builtin_pwd (void)

  Retrieves the current working directory.
- void execute_builtin_pwd (t_shell *shell)

  Executes the built-in pwd command.

4.17.1 Function Documentation

4.17.1.1 builtin_pwd()

```
char* builtin_pwd (
            void  )
```

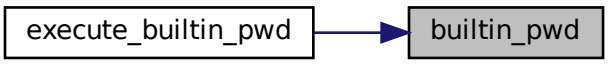
Retrieves the current working directory.

This function uses the getcwd() function to obtain the current working directory and returns it as a dynamically allocated string. The caller is responsible for freeing the allocated memory.

Returns

A pointer to the current working directory string, or NULL if an error occurs.

Here is the caller graph for this function:



4.17.1.2 execute_builtin_pwd()

```
void execute_builtin_pwd (
            t_shell * shell )
```

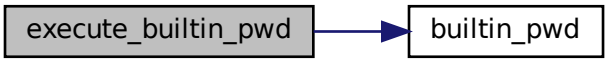
Executes the built-in pwd command.

This function retrieves the current working directory using the builtin_pwd() function. If the directory is successfully retrieved, it prints the directory path to the standard output and sets the shell's exit code to 0. If the directory retrieval fails, it prints an error message using perror and sets the shell's exit code to 1.

Parameters

shell	A pointer to the shell structure containing the exit code.

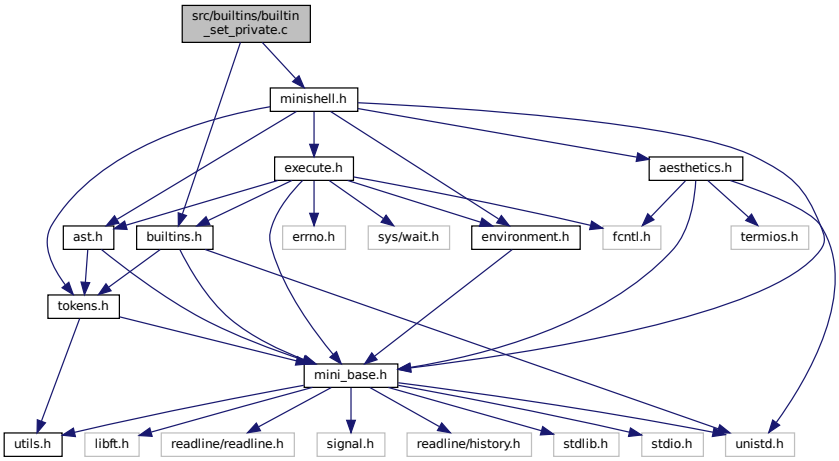
Here is the call graph for this function:



4.18 src/builtins/builtin_set_private.c File Reference

```
#include <builtins.h>
#include <minishell.h>
```

Include dependency graph for builtin_set_private.c:



Functions

- void builtin_set_private (t_shell *shell, t_ast_node *node)

  Sets a private environment variable in the shell.

4.18.1 Function Documentation

4.18.1.1 builtin_set_private()

```
void builtin_set_private (
            t_shell * shell,
            t_ast_node * node )
```

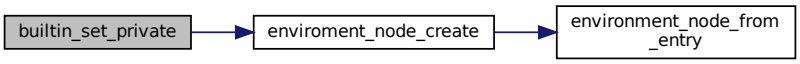
Sets a private environment variable in the shell.

This function takes a shell structure and an AST node, and sets a private environment variable based on the arguments provided in the AST node. If the required arguments are not present, it sets the shell's exit code to 1 and returns. Otherwise, it creates a new environment entry and updates the shell's environment.

Parameters

shell	A pointer to the shell structure.
node	A pointer to the AST node containing the arguments.

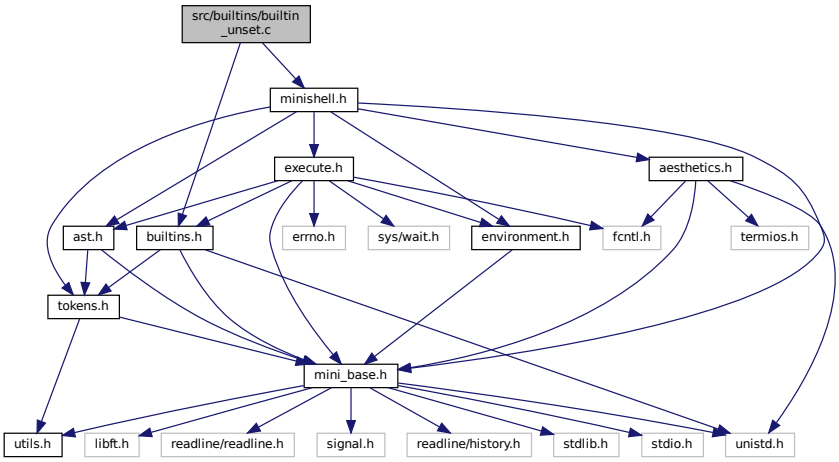
Here is the call graph for this function:



4.19 src/builtins/builtin_unset.c File Reference

```
#include <builtins.h>
#include <minishell.h>
```

Include dependency graph for builtin_unset.c:



Functions

- void builtin_unset (t_shell *shell, t_ast_node *node)

  Unsets environment variables in the shell.

4.19.1 Function Documentation

4.19.1.1 builtin_unset()

```
void builtin_unset (
            t_shell * shell,
            t_ast_node * node )
```

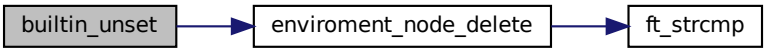
Unsets environment variables in the shell.

This function removes environment variables specified in the arguments from the shell's environment. If no arguments are provided, it prints an error message indicating that not enough arguments were given.

Parameters

shell	A pointer to the shell structure.
node	A pointer to the AST node containing the command and its arguments.

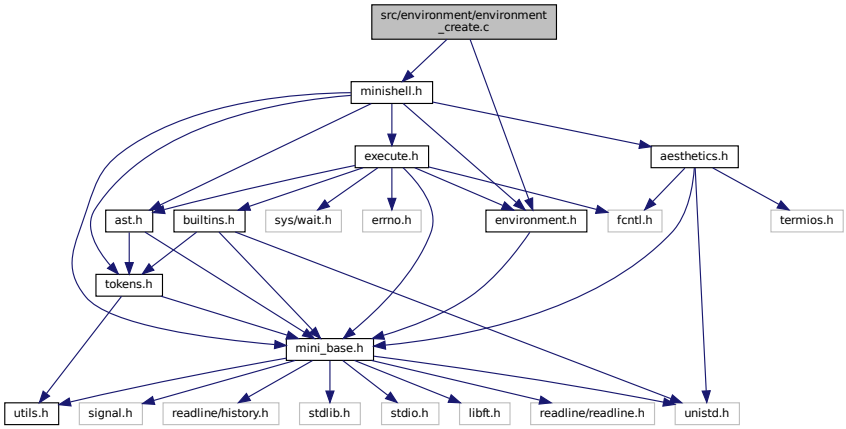
Here is the call graph for this function:



4.20 src/environment/environment_create.c File Reference

```
#include <environment.h>
#include <minishell.h>
```

Include dependency graph for environment_create.c:



Functions

- t_environment_node ∗ enviroment_node_create (const char ∗entry, t_shell ∗shell, t_bool is_private)

  Creates an environment node from an entry string and adds it to the shell's environment list.
- t_environment_node ∗ environment_list_initialize (const char ∗∗envp)

  Initializes an environment list from an array of environment variable strings.

4.20.1 Function Documentation

4.20.1.1 enviroment_node_create()

```
t_environment_node* enviroment_node_create (
            const char * entry,
            t_shell * shell,
            t_bool is_private )
```

Creates an environment node from an entry string and adds it to the shell's environment list.

This function uses environment_node_from_entry to parse the entry string into a node. If successful, it inserts this node into the shell's environment list using _environment_node_insert. It returns the created node, or NULL if parsing fails.

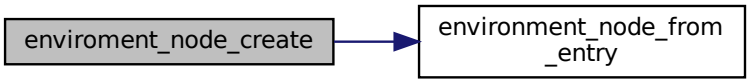
Parameters

entry	The entry string to process.
shell	A pointer to the shell context containing the environment list.

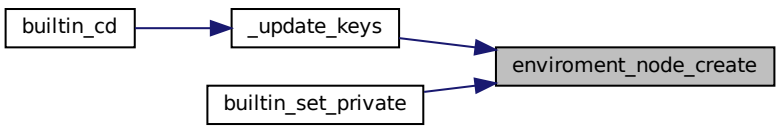
Returns

t_environment_node* A pointer to the newly created and inserted node, or NULL.

Here is the call graph for this function:



Here is the caller graph for this function:



4.20.1.2 environment_list_initialize()

```
t_environment_node* environment_list_initialize (
            const char ** envp )
```

Initializes an environment list from an array of environment variable strings.

This function processes each string in the provided array using environment_node_from_entry to create nodes. Each valid node is inserted into the environment list. It returns the head of the newly created list, which may be empty if all entries are invalid.

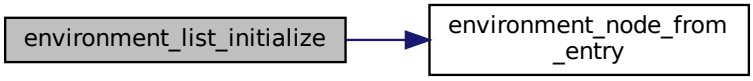
Parameters

envp	An array of strings representing environment variables.

Returns

t_environment_node* The head of the environment list, or NULL if no nodes were created.

Here is the call graph for this function:

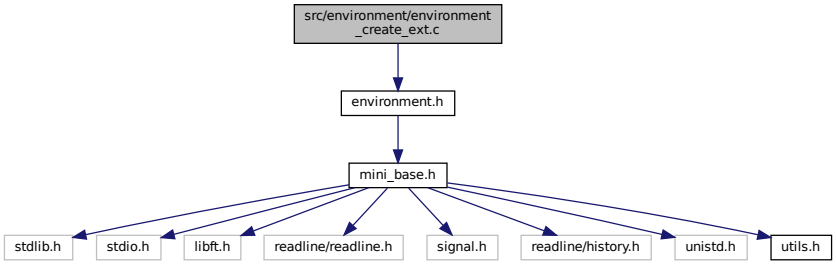


Here is the caller graph for this function:

4.21 src/environment/environment_create_ext.c File Reference

```
#include <environment.h>
```

Include dependency graph for environment_create_ext.c:



Functions

- t_environment_node * environment_node_from_entry (const char *entry, t_bool is_private)

  Creates a new environment node from an entry string.

4.21.1 Function Documentation

4.21.1.1 environment_node_from_entry()

```
t_environment_node* environment_node_from_entry (
            const char * entry,
            t_bool is_private )
```

Creates a new environment node from an entry string.

This function takes an entry string, trims any surrounding double quotes, and splits it into key-value pairs using the '=' delimiter. It validates that exactly two parts exist after splitting. If invalid, it cleans up resources and returns NULL. Otherwise, it initializes a new node with the parsed values.

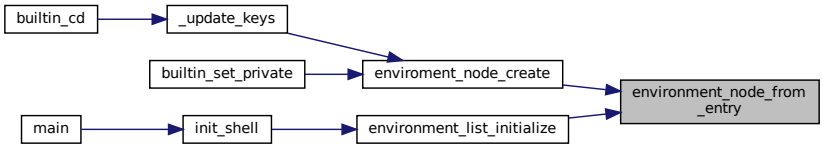
Parameters

entry	The entry string to parse.

Returns

t_environment_node* A pointer to the newly created node, or NULL if parsing failed.

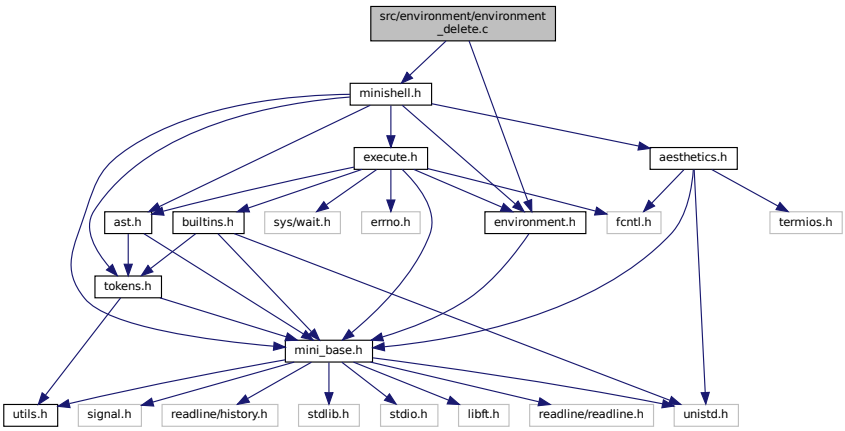
Here is the caller graph for this function:



4.22 src/environment/environment_delete.c File Reference

```
#include <environment.h>
#include <minishell.h>
```

Include dependency graph for environment_delete.c:



Functions

- void enviroment_node_delete (const char *key, t_shell *shell)

  Deletes an environment node with a specified key from the shell's environment list.
- void environment_list_clear (t_shell *shell)

  Clears all nodes in the shell's environment list.

4.22.1 Function Documentation

4.22.1.1 enviroment_node_delete()

```
void enviroment_node_delete (
            const char * key,
            t_shell * shell )
```

Deletes an environment node with a specified key from the shell's environment list.

This function searches for a node with the given key in the shell's environment list. If found, it removes the node from the list and frees its resources. If not found, or if inputs are invalid, it does nothing.

Parameters

key	The key of the node to delete.
shell	A pointer to the shell context containing the environment list.

Here is the call graph for this function:

Here is the caller graph for this function:

4.22.1.2 environment_list_clear()

```
void environment_list_clear (
            t_shell * shell )
```

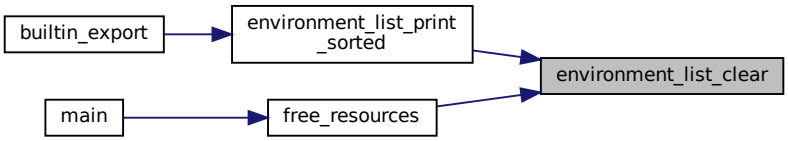
Clears all nodes in the shell's environment list.

This function iterates through each node in the environment list, deletes them one by one using _environment_↩ node_delete, and updates the head pointer to NULL. It ensures that all resources are properly freed and the list is completely cleared.

Parameters

shell	A pointer to the shell context containing the environment list.

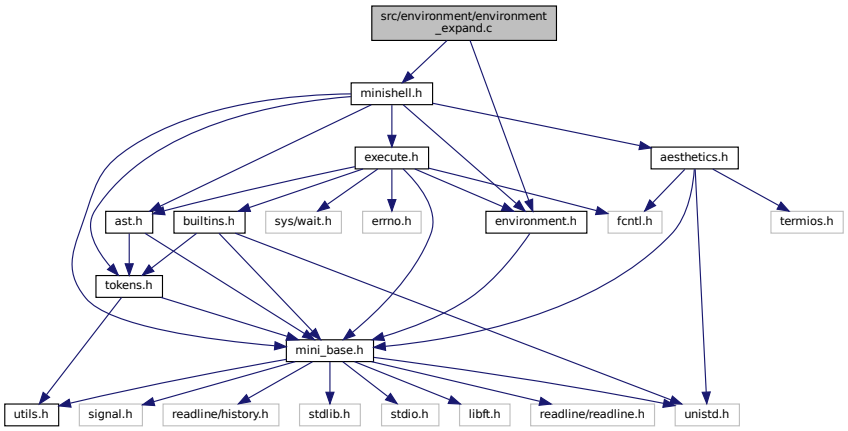
Here is the caller graph for this function:



4.23 src/environment/environment_expand.c File Reference

```
#include <environment.h>
#include <minishell.h>
```

Include dependency graph for environment_expand.c:



Functions

- char ∗ env_value_expand (t_shell ∗shell, char ∗key)

  Expands the value of an environment variable based on the given key.

4.23.1 Function Documentation

4.23.1.1 env_value_expand()

```
char* env_value_expand (
            t_shell * shell,
            char * key )
```

Expands the value of an environment variable based on the given key.

This function takes a key and returns the corresponding environment variable's value. It handles different cases based on the first character of the key:

- If the key starts with '$', it skips the '$' and looks up the environment variable.
- If the key starts with '"', it expands multiple variables within the key.
- If the key starts with '\', it trims the quotes from the key.
- Otherwise, it returns a duplicate of the key.

Additionally, if the key is "?", it returns the shell's exit code as a string.

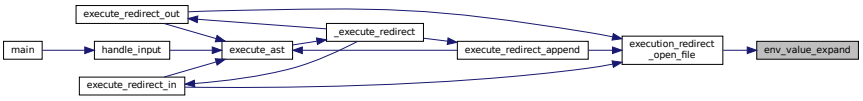
Parameters

shell	A pointer to the shell structure containing environment variables and exit code.
key	The key of the environment variable to expand.

Returns

A dynamically allocated string containing the expanded value, or NULL if the key is invalid or the value is not found.

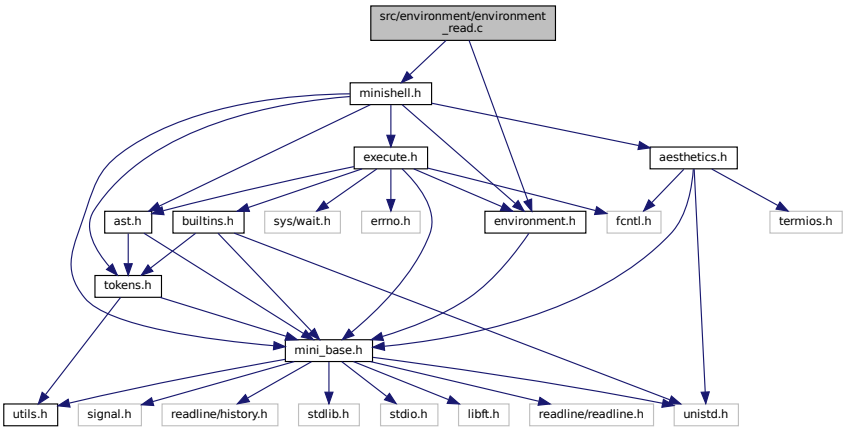
Here is the caller graph for this function:



4.24 src/environment/environment_read.c File Reference

```
#include <environment.h>
#include <minishell.h>
```

Include dependency graph for environment_read.c:



Functions

- char * environment_list_read (const char *key, t_shell *shell)

  Reads the value associated with a given key from the environment list.
- t_environment_node * environment_list_read_node (const char *key, t_shell *shell)

  Reads a node from the environment list based on the given key.
- void environment_list_print (t_shell *shell, t_bool is_export)

  Prints the environment variables in the shell.
- void environment_list_print_sorted (t_shell *shell)

  Prints the environment variables in sorted order.

4.24.1 Function Documentation

4.24.1.1 environment_list_print()

```
void environment_list_print (
            t_shell * shell,
            t_bool is_export )
```

Prints the environment variables in the shell.

This function iterates through the linked list of environment variables and prints each variable in the format specified by the `is_export` flag.

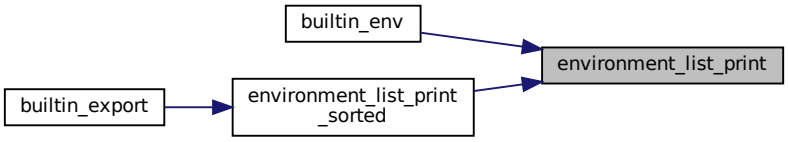
Parameters

shell	Pointer to the shell structure containing the environment list.
is_export	Boolean flag indicating the format of the output: • If true, prints variables in the format used by the `export` command. • If false, prints variables in the standard key=value format.

Returns

void

Here is the caller graph for this function:



4.24.1.2 environment_list_print_sorted()

```
void environment_list_print_sorted (
            t_shell * shell )
```

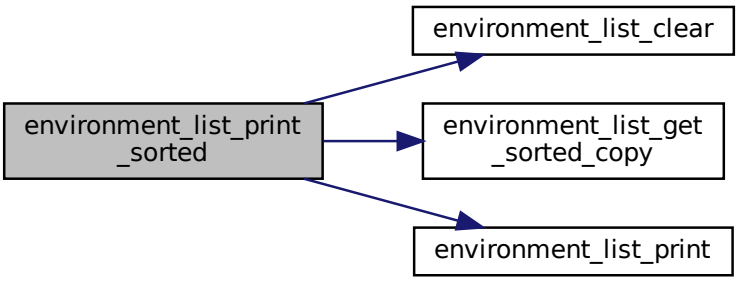
Prints the environment variables in sorted order.

This function creates a temporary copy of the environment variables, sorts them, and then prints them. After printing, it clears the temporary environment list to free up memory.

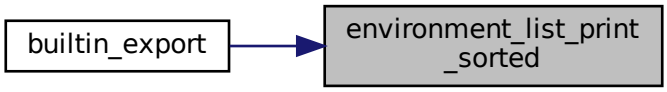
Parameters

shell	A pointer to the shell structure containing the environment list.

Here is the call graph for this function:



Here is the caller graph for this function:



4.24.1.3 environment_list_read()

```
char* environment_list_read (
            const char * key,
            t_shell * shell )
```

Reads the value associated with a given key from the environment list.

This function traverses the linked list of environment variables stored in the shell structure and returns the value associated with the specified key.

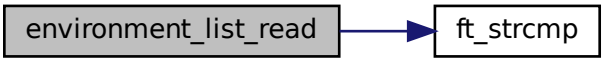
Parameters

key	The key to search for in the environment list.
shell	A pointer to the shell structure containing the environment list.

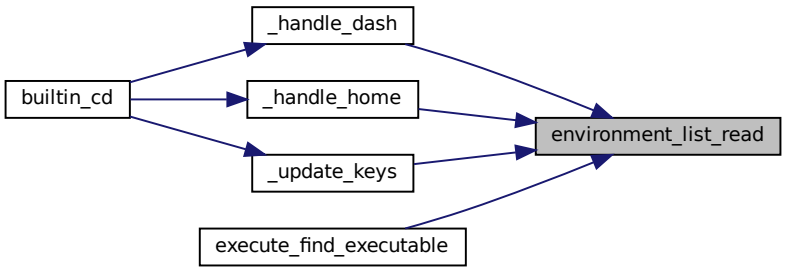
Returns

The value associated with the specified key, or an empty string if the key is not found.

Here is the call graph for this function:



Here is the caller graph for this function:



4.24.1.4 environment_list_read_node()

```
t_environment_node* environment_list_read_node (
            const char * key,
            t_shell * shell )
```

Reads a node from the environment list based on the given key.

This function searches through the environment list in the shell structure to find a node that matches the provided key. If a matching node is found, it is returned. Otherwise, the function returns NULL.

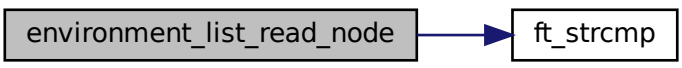
Parameters

key	The key to search for in the environment list.
shell	A pointer to the shell structure containing the environment list.

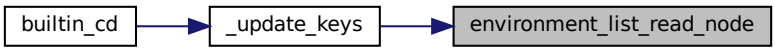
Returns

t_environment_node* A pointer to the matching environment node, or NULL if not found.

Here is the call graph for this function:



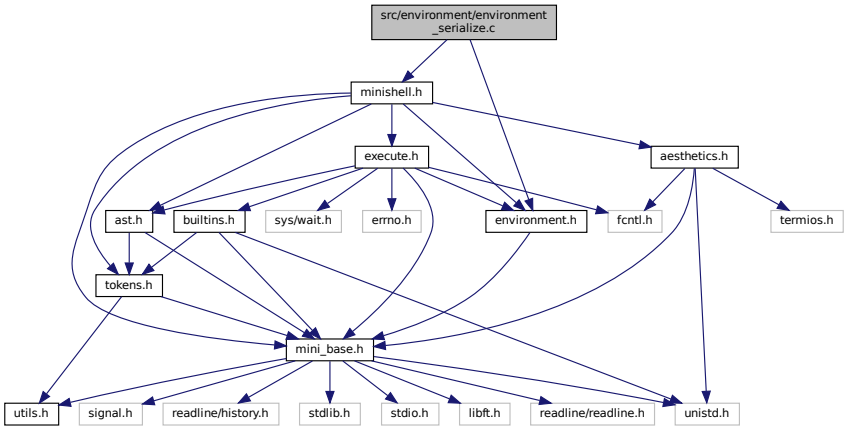
Here is the caller graph for this function:



4.25 src/environment/environment_serialize.c File Reference

```
#include <environment.h>
#include <minishell.h>
```

Include dependency graph for environment_serialize.c:



Functions

- char ** environment_serialize (t_shell *shell)

  Serializes the environment variables from the shell into an array of strings.
- void environment_serialized_list_clear (char **list)

  Frees a list of strings and the list itself.

4.25.1 Function Documentation

4.25.1.1 environment_serialize()

```
char** environment_serialize (
            t_shell * shell )
```

Serializes the environment variables from the shell into an array of strings.

This function takes the environment variables stored in the shell's environment list and serializes them into an array of strings, where each string is in the format "key=value". Private environment variables (marked by `is_private`) are excluded from the serialization.

Parameters

shell	A pointer to the shell structure containing the environment list.

Returns

A NULL-terminated array of strings representing the serialized environment variables. Returns NULL if there are no environment variables or if memory allocation fails.

4.25.1.2 environment_serialized_list_clear()

```
void environment_serialized_list_clear (
            char ** list )
```

Frees a list of strings and the list itself.

This function iterates through a list of strings, freeing each string, and then frees the list itself.

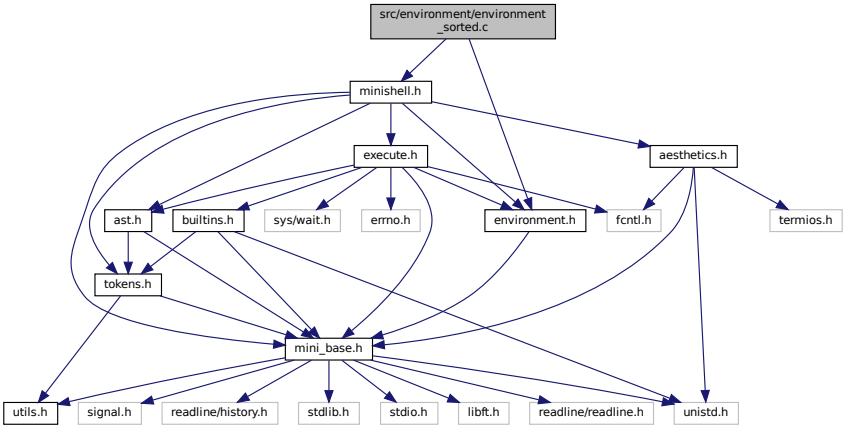
Parameters

list	A null-terminated array of strings to be freed.

4.26 src/environment/environment_sorted.c File Reference

```
#include <environment.h>
#include <minishell.h>
```

Include dependency graph for environment_sorted.c:



Functions

- t_environment_node * environment_list_get_sorted_copy (t_environment_node *original)

  Creates a sorted copy of the environment list.

4.26.1 Function Documentation

4.26.1.1 environment_list_get_sorted_copy()

```
t_environment_node* environment_list_get_sorted_copy (
            t_environment_node * original )
```

Creates a sorted copy of the environment list.

This function takes an unsorted linked list of environment variables and returns a new linked list that is sorted by the keys of the environment variables. The original list remains unchanged.

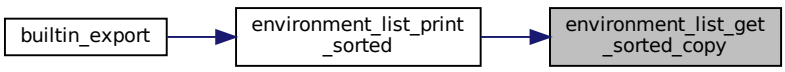
Parameters

original	A pointer to the head of the original unsorted environment list.

Returns

A pointer to the head of the new sorted environment list.

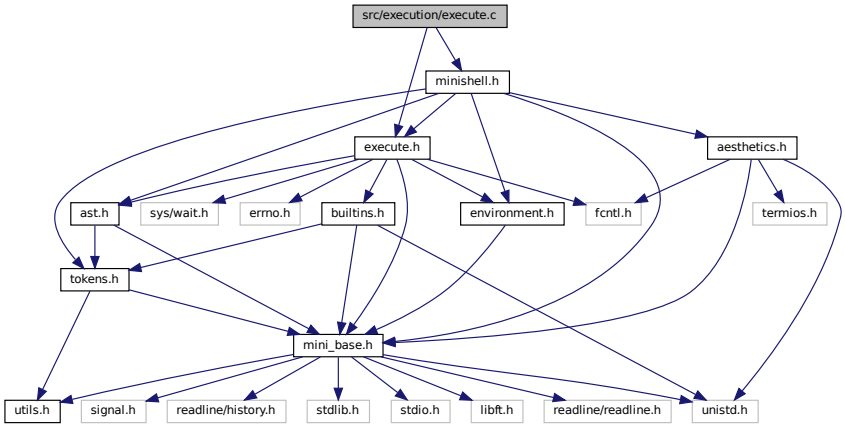
Here is the caller graph for this function:



4.27 src/execution/execute.c File Reference

```
#include <execute.h>
#include <minishell.h>
```

Include dependency graph for execute.c:



Functions

- void _execute_redirect (t_shell ∗shell, t_ast_node ∗node)

  Executes the appropriate redirection based on the type of the AST node.
- void execute_ast (t_shell ∗shell, t_ast_node ∗node)

  Executes the abstract syntax tree (AST) node.

4.27.1 Function Documentation

4.27.1.1 _execute_redirect()

```
void _execute_redirect (
            t_shell * shell,
            t_ast_node * node )
```

Executes the appropriate redirection based on the type of the AST node.

This function handles different types of redirections by calling the corresponding redirection function based on the type of the AST node provided.

Parameters

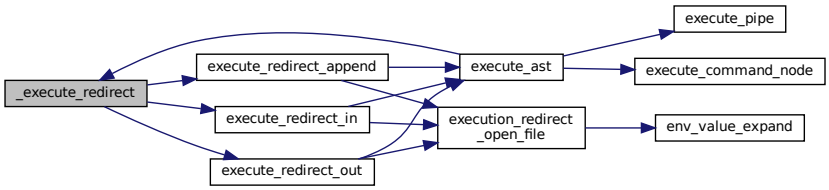
shell	A pointer to the shell structure containing the shell's state.
node	A pointer to the AST node representing the redirection operation.

The function supports the following redirection types:

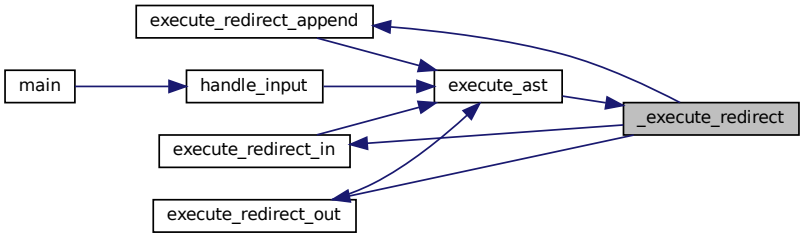
- TOKEN_REDIRECT_IN: Calls execute_redirect_in to handle input redirection.

- TOKEN_REDIRECT_OUT: Calls execute_redirect_out to handle output redirection.
- TOKEN_APPEND: Calls execute_redirect_append to handle output redirection in append mode.

Here is the call graph for this function:



Here is the caller graph for this function:



4.27.1.2 execute_ast()

```
void execute_ast (
            t_shell * shell,
            t_ast_node * node )
```

Executes the abstract syntax tree (AST) node.

This function takes a shell context and an AST node, and executes the node based on its type. The function handles different types of nodes such as command nodes, pipe nodes, and various types of redirection nodes.

Parameters

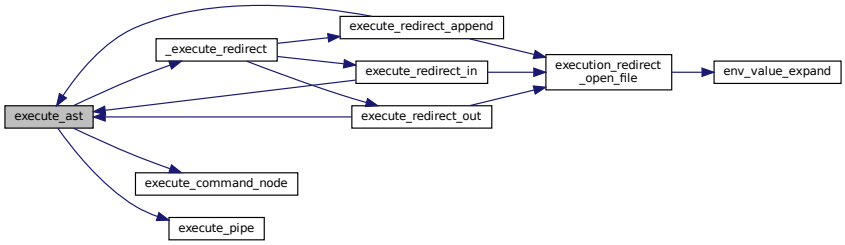
shell	A pointer to the shell context.
node	A pointer to the AST node to be executed.

Note

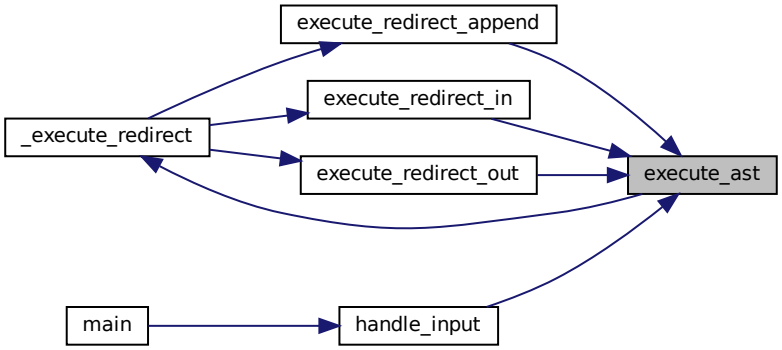
If the node is NULL, the function returns immediately.

The function delegates the execution to specific functions based on the type of the node.

Here is the call graph for this function:



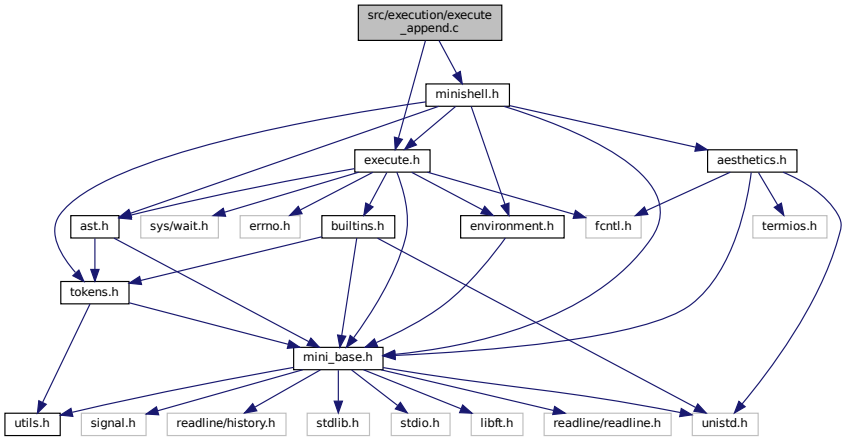
Here is the caller graph for this function:



4.28 src/execution/execute_append.c File Reference

```
#include <execute.h>
#include <minishell.h>
```

Include dependency graph for execute_append.c:



Functions

- void execute_redirect_append (t_shell *shell, t_ast_node *node)

4.28.1 Function Documentation

4.28.1.1 execute_redirect_append()

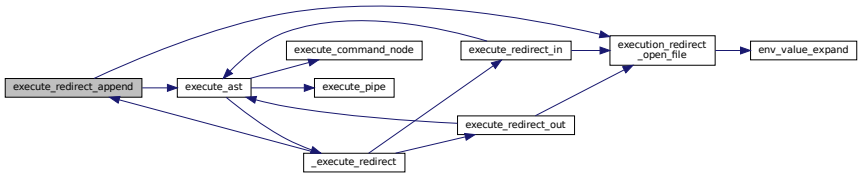
```
void execute_redirect_append (
            t_shell * shell,
            t_ast_node * node )
```

execute_redirect_append - Executes a command with output redirected to a file in append mode. @shell: Pointer to the shell structure containing environment variables and state. @node: Pointer to the AST node representing the command and redirection.

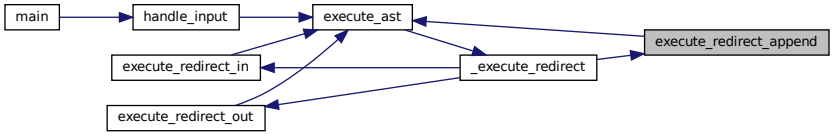
This function performs the following steps:

1. Duplicates the current STDOUT file descriptor to save the original STDOUT.
2. Expands the file name using environment variable expansion.
3. Opens the file in append mode, creating it if it does not exist.
4. Redirects STDOUT to the opened file.
5. Executes the command represented by the left child of the AST node.
6. Restores the original STDOUT.
7. Closes the file descriptors.

If the file cannot be opened, the function prints an error message and exits. Here is the call graph for this function:



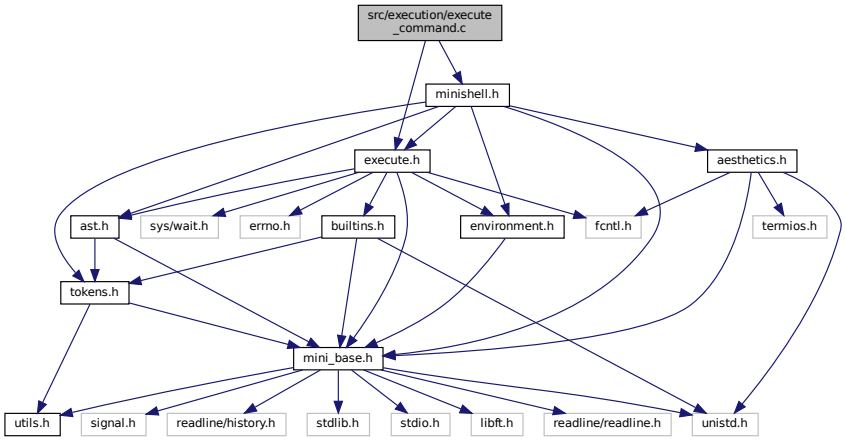
Here is the caller graph for this function:



4.29 src/execution/execute_command.c File Reference

```
#include <execute.h>
#include <minishell.h>
```

Include dependency graph for execute_command.c:



Functions

- void execute_command_node (t_shell *shell, t_ast_node *node)

  Executes a command node in the shell's abstract syntax tree (AST).

4.29.1 Function Documentation

4.29.1.1 execute_command_node()

```
void execute_command_node (
            t_shell * shell,
            t_ast_node * node )
```

Executes a command node in the shell's abstract syntax tree (AST).

This function determines if the command node represents a built-in command or a more complex command. If it is a built-in command, it executes it directly. Otherwise, it forks a new process to execute the complex command.

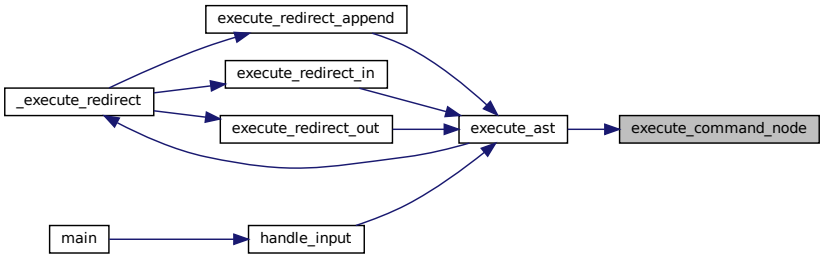
Parameters

shell	A pointer to the shell structure containing the shell's state.
node	A pointer to the AST node representing the command to be executed.

Note

If the command is not a built-in, the function forks a new process. The parent process waits for the child process to complete and updates the shell's exit code based on the child's exit status.

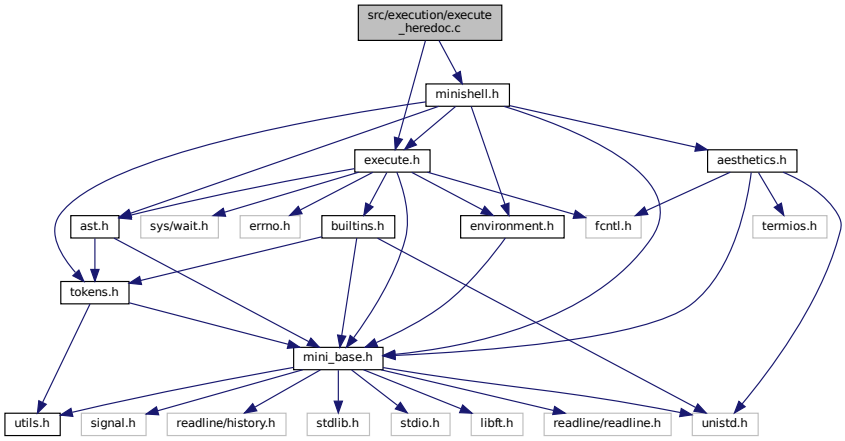
Here is the caller graph for this function:



4.30 src/execution/execute_heredoc.c File Reference

```
#include <execute.h>
#include <minishell.h>
```

Include dependency graph for execute_heredoc.c:



Functions

- void process_heredoc_input (char *temp_filename, const char *delimiter)

  Processes input for a heredoc and writes it to a temporary file.
- void execute_preprocess_heredocs (t_ast_node *node)

  Preprocesses heredoc nodes in the abstract syntax tree (AST).

4.30.1 Function Documentation

4.30.1.1 execute_preprocess_heredocs()

```
void execute_preprocess_heredocs (
            t_ast_node * node )
```

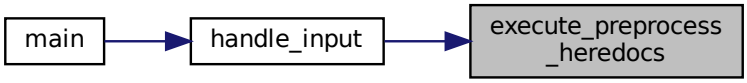
Preprocesses heredoc nodes in the abstract syntax tree (AST).

This function recursively traverses the AST and processes nodes of type TOKEN_HEREDOC. For each such node, it trims the quotes from the delimiter, creates a temporary file name, processes the heredoc input, and updates the node type to TOKEN_REDIRECT_IN. It also updates the node's arguments to point to the temporary file name.

Parameters

node	A pointer to the current AST node being processed.

Here is the caller graph for this function:



4.30.1.2 process_heredoc_input()

```
void process_heredoc_input (
            char * temp_filename,
            const char * delimiter )
```

Processes input for a heredoc and writes it to a temporary file.

This function reads lines from the standard input until a line matching the specified delimiter is encountered. Each line is written to the specified temporary file. The temporary file is created if it does not exist, and truncated if it does.

Parameters

temp_filename	The name of the temporary file to write the heredoc input to.
delimiter	The delimiter string that indicates the end of the heredoc input.

Note

If temp_filename is NULL or if the file cannot be opened, the function will exit with EXIT_FAILURE.

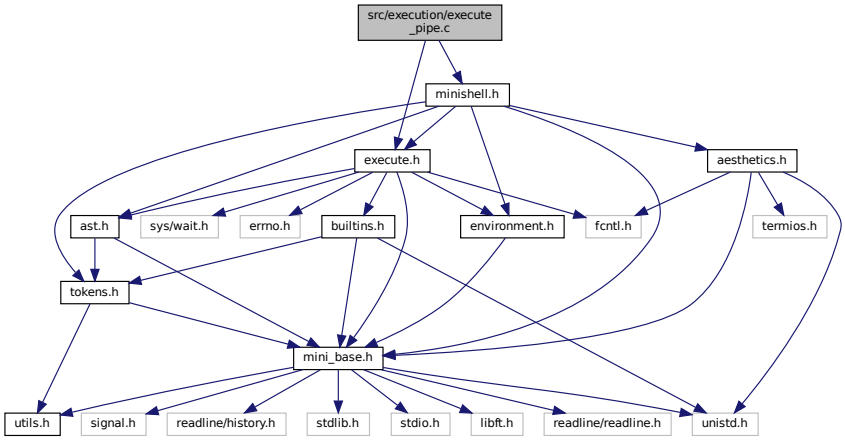
Here is the call graph for this function:

4.31 src/execution/execute_pipe.c File Reference

```
#include <execute.h>
```

```
#include <minishell.h>
```

Include dependency graph for execute_pipe.c:



Functions

- void execute_pipe (t_shell *shell, t_ast_node *node)

  Executes a command pipeline by forking two child processes.

4.31.1 Function Documentation

4.31.1.1 execute_pipe()

```
void execute_pipe (
            t_shell * shell,
            t_ast_node * node )
```

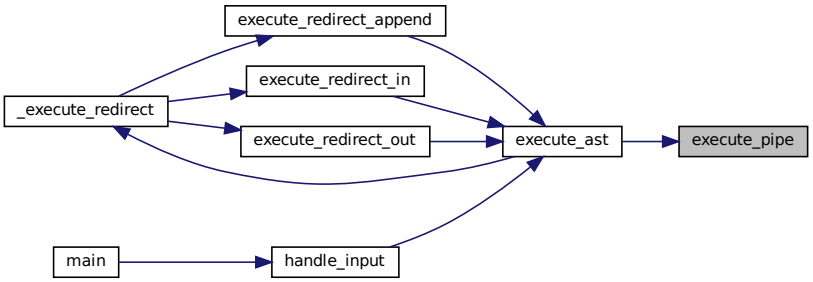
Executes a command pipeline by forking two child processes.

This function creates a pipe and forks two child processes to execute the left and right sides of a pipeline. The left child process writes to the pipe, and the right child process reads from the pipe. The parent process waits for both child processes to complete and sets the shell's exit code based on the status of the right child process.

Parameters

shell	A pointer to the shell structure.
node	A pointer to the abstract syntax tree node representing the pipeline.

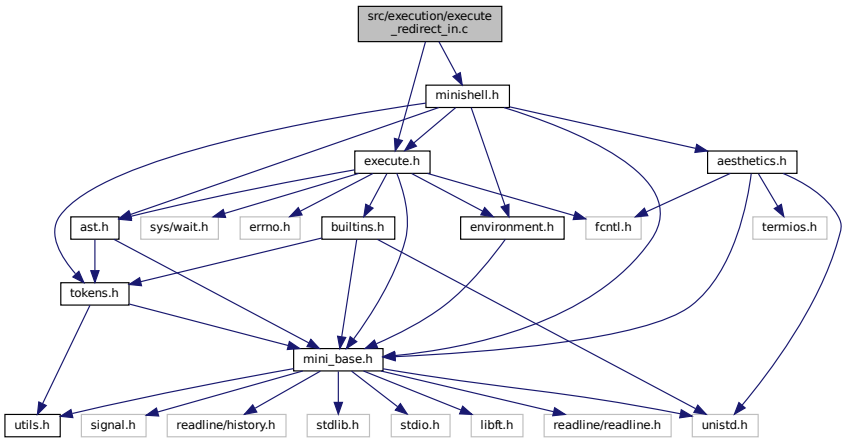
Here is the caller graph for this function:



4.32 src/execution/execute_redirect_in.c File Reference

```
#include <execute.h>
#include <minishell.h>
```

Include dependency graph for execute_redirect_in.c:



Functions

- void execute_redirect_in (t_shell *shell, t_ast_node *node)

  Executes a command with input redirection.

4.32.1 Function Documentation

4.32.1.1 execute_redirect_in()

```
void execute_redirect_in (
            t_shell * shell,
            t_ast_node * node )
```

Executes a command with input redirection.

This function handles the execution of a command where the input is redirected from a file. It temporarily replaces the standard input (stdin) with the contents of the specified file, executes the command, and then restores the original stdin.

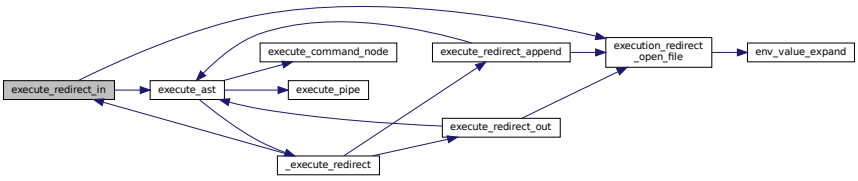
Parameters

shell	Pointer to the shell structure containing environment and state information.
node	Pointer to the AST node representing the command and its redirection.

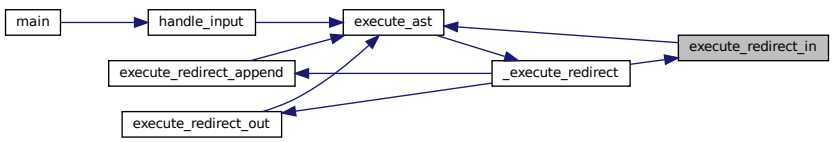
The function performs the following steps:

1. Duplicates the current stdin file descriptor to save it.
2. Expands the file name using environment variable expansion.
3. Opens the file for reading.
4. If the file cannot be opened, prints an error message and exits.
5. Redirects stdin to the opened file.
6. Executes the command represented by the left child of the AST node.
7. Restores the original stdin.
8. Closes the duplicated stdin file descriptor.

Here is the call graph for this function:



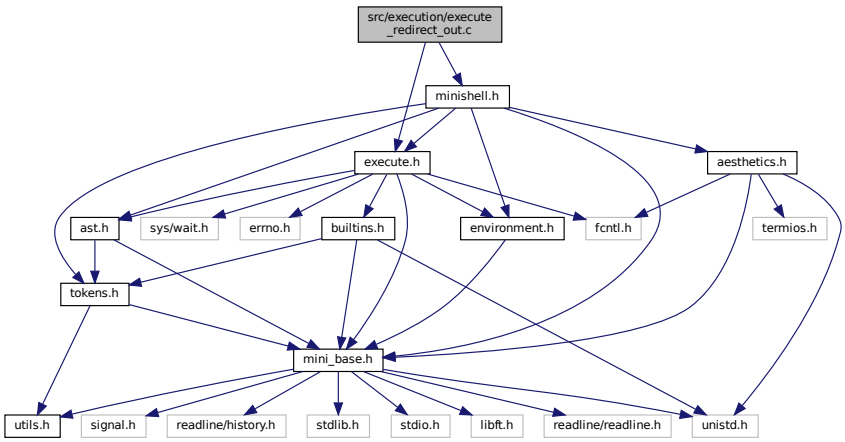
Here is the caller graph for this function:



4.33 src/execution/execute_redirect_out.c File Reference

```
#include <execute.h>
#include <minishell.h>
```

Include dependency graph for execute_redirect_out.c:



Functions

- void execute_redirect_out (t_shell *shell, t_ast_node *node)

  Executes a command with output redirected to a file.

4.33.1 Function Documentation

4.33.1.1 execute_redirect_out()

```
void execute_redirect_out (
            t_shell * shell,
            t_ast_node * node )
```

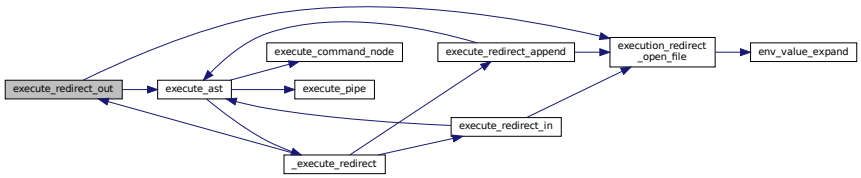
Executes a command with output redirected to a file.

This function handles the redirection of the standard output (stdout) to a file specified in the abstract syntax tree (AST) node. It first duplicates the current stdout file descriptor to restore it later. Then, it expands the file name using environment variable expansion, opens the file with write permissions, and redirects stdout to this file. After executing the command represented by the left child of the AST node, it restores the original stdout file descriptor.

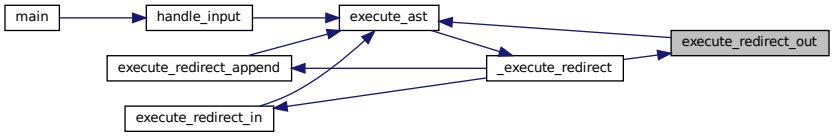
Parameters

shell	Pointer to the shell structure containing the environment and state.
node	Pointer to the AST node representing the redirection operation.

Here is the call graph for this function:



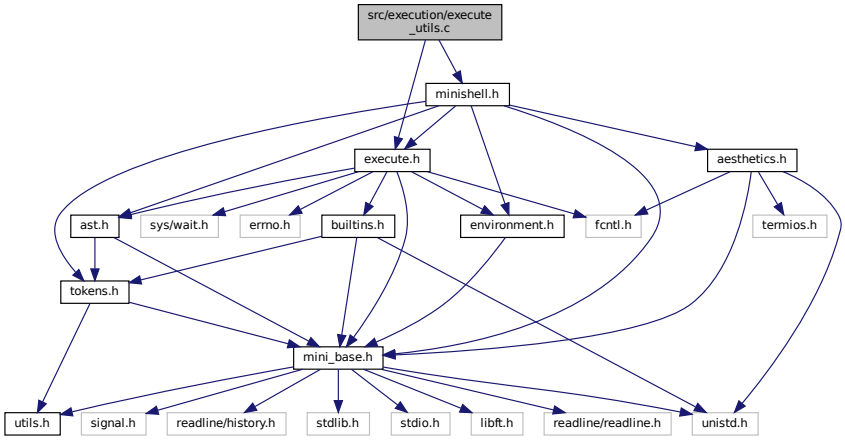
Here is the caller graph for this function:



4.34 src/execution/execute_utils.c File Reference

```
#include <execute.h>
#include <minishell.h>
```

Include dependency graph for execute_utils.c:



Functions

- char ∗ execute_find_executable (char ∗command, t_shell ∗shell)

  Finds the full path of an executable command.
- int execution_redirect_open_file (t_shell ∗shell, t_ast_node ∗node)

  Opens a file for redirection during command execution.

4.34.1 Function Documentation

4.34.1.1 execute_find_executable()

```
char* execute_find_executable (
            char * command,
            t_shell * shell )
```

Finds the full path of an executable command.

This function searches for the given command in the directories specified by the PATH environment variable. If the command is an absolute path or a relative path starting with "./", it checks if the command is executable.

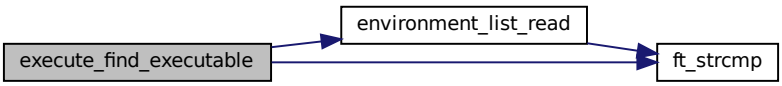
Parameters

command	The command to find the executable path for.
shell	The shell structure containing environment variables.

Returns

A dynamically allocated string containing the full path of the executable if found, or NULL if the command is not found or not executable.

Here is the call graph for this function:



4.34.1.2 execution_redirect_open_file()

```
int execution_redirect_open_file (
            t_shell * shell,
            t_ast_node * node )
```

Opens a file for redirection during command execution.

This function expands the file name using environment variables, opens the file with the appropriate mode, and returns the file descriptor. If the file cannot be opened, the function will terminate the program with an exit status of failure.

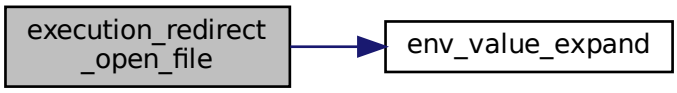
Parameters

shell	A pointer to the shell structure containing environment variables.
node	A pointer to the AST node containing the file redirection information.

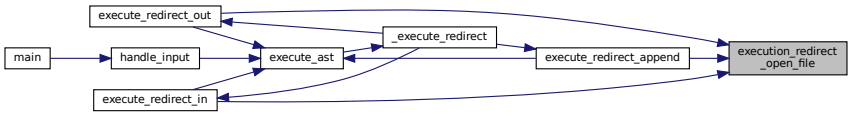
Returns

The file descriptor of the opened file.

Here is the call graph for this function:



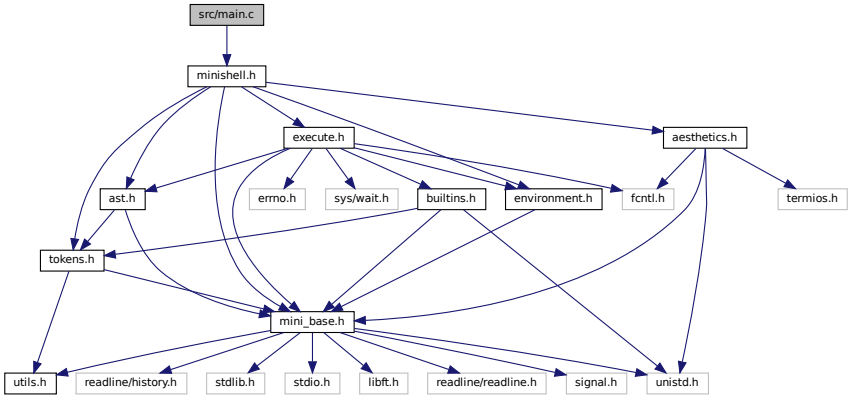
Here is the caller graph for this function:



4.35 src/main.c File Reference

`#include <minishell.h>`

Include dependency graph for main.c:



Functions

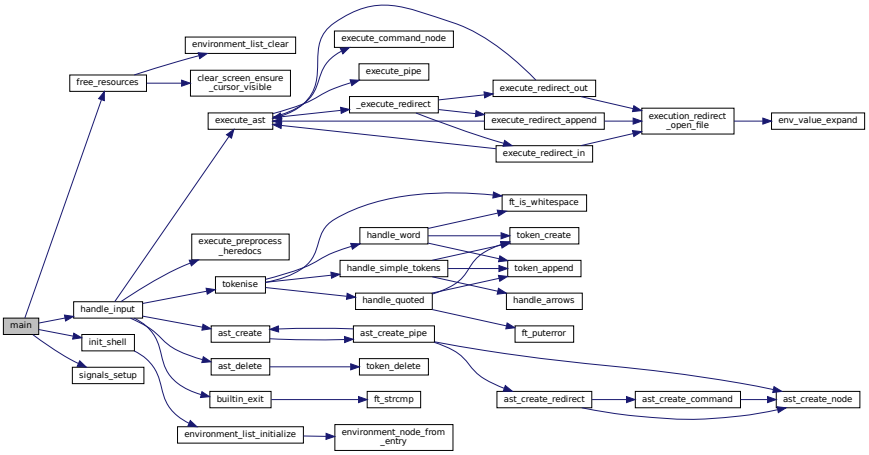
- int main (int ac, const char **av, const char **envp)

4.35.1 Function Documentation

4.35.1.1 main()

```
int main (
            int ac,
            const char ** av,
            const char ** envp )
```

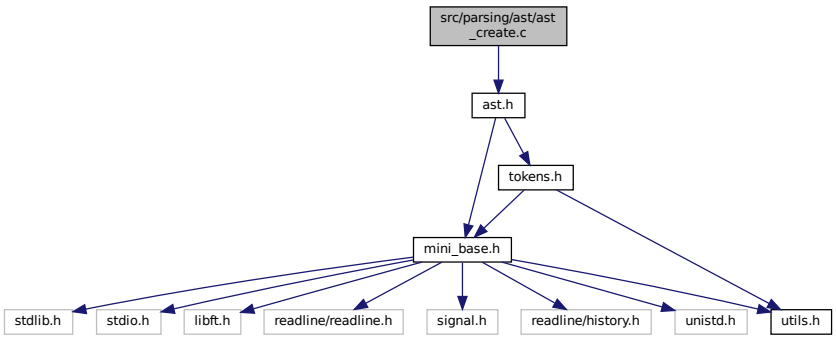
Here is the call graph for this function:



4.36 src/parsing/ast/ast_create.c File Reference

```
#include <ast.h>
```

Include dependency graph for ast_create.c:



Functions

- t_ast_node * ast_create_node (t_token_node *token_node)

  Creates a new AST node.
- t_ast_node * ast_create (t_token_node **list)

  Creates an abstract syntax tree (AST) from a list of tokens.

4.36.1 Function Documentation

4.36.1.1 ast_create()

```
t_ast_node* ast_create (
            t_token_node ** list )
```

Creates an abstract syntax tree (AST) from a list of tokens.

This function takes a list of tokens and constructs an AST node from it. If the list is empty or NULL, the function returns NULL. Otherwise, it delegates the creation of the AST to the `ast_create_pipe` function.

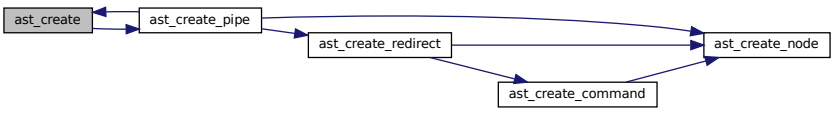
Parameters

list	A pointer to the list of tokens to be parsed into an AST.

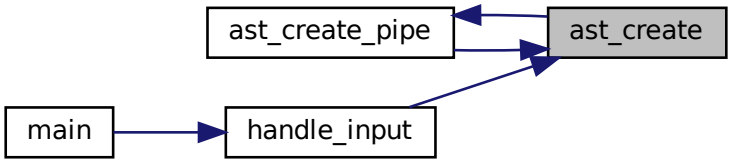
Returns

A pointer to the created AST node, or NULL if the list is empty or NULL.

Here is the call graph for this function:



Here is the caller graph for this function:



4.36.1.2 ast_create_node()

```
t_ast_node* ast_create_node (
            t_token_node * token_node )
```

Creates a new AST node.

This function allocates memory for a new `t_ast_node` structure and initializes its fields. The `type` parameter specifies the type of the node, while the `value` parameter specifies the value associated with the node. If `value` is `NULL`, the `value` field of the node will be set to `NULL`.

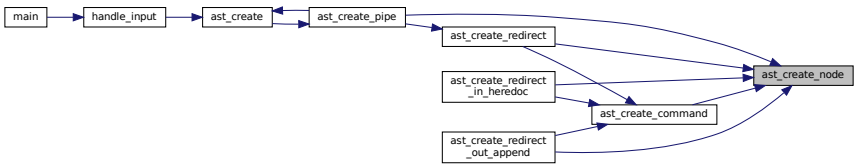
Parameters

type	The type of the AST node.
value	The value associated with the AST node.

Returns

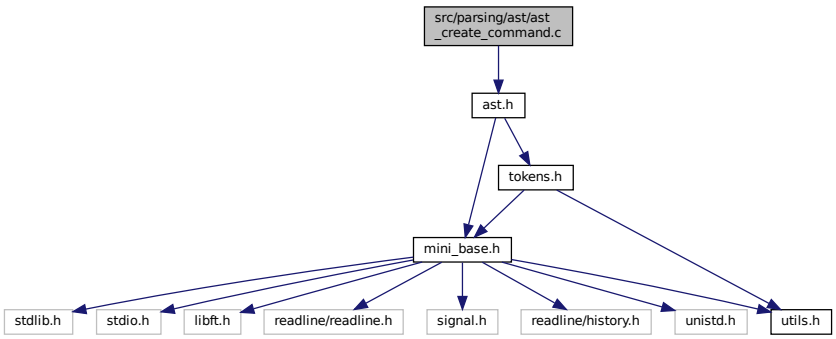
A pointer to the newly created `t_ast_node` structure, or `NULL` if memory allocation fails.

Here is the caller graph for this function:



4.37 src/parsing/ast/ast_create_command.c File Reference

`#include <ast.h>`
Include dependency graph for ast_create_command.c:



Functions

- t_ast_node * ast_create_command (t_token_node **list)

4.37.1 Function Documentation

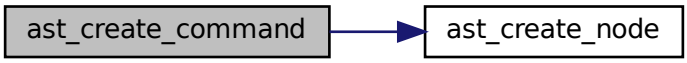
4.37.1.1 ast_create_command()

```
t_ast_node* ast_create_command (
            t_token_node ** list )
```

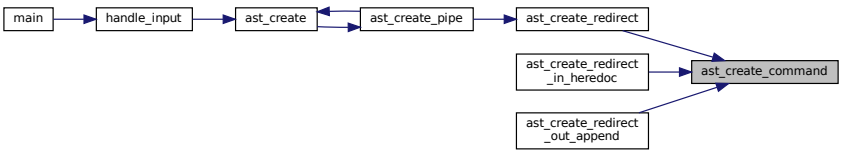
ast_parse_command - Parses a command from a token list and creates an AST node. @list: A double pointer to the head of the token list.

This function checks if the provided token list is valid and if the head of the list is of type TOKEN_STRING. If so, it creates an AST node from the head token, advances the list to the next token, and returns the created AST node. If the list is invalid or the head token is not of type TOKEN_STRING, it returns NULL.

Return: A pointer to the created AST node, or NULL if the list is invalid or the head token is not of type TOKEN_↩ STRING. Here is the call graph for this function:



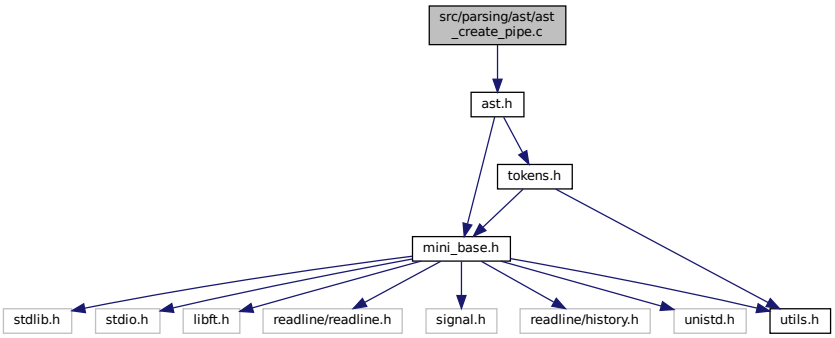
Here is the caller graph for this function:



4.38 src/parsing/ast/ast_create_pipe.c File Reference

```
#include <ast.h>
```

Include dependency graph for ast_create_pipe.c:



Functions

- t_ast_node * ast_create_pipe (t_token_node **list)

4.38.1 Function Documentation

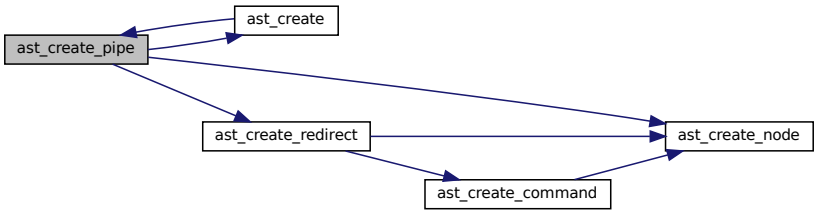
4.38.1.1 ast_create_pipe()

```
t_ast_node* ast_create_pipe (
            t_token_node ** list )
```

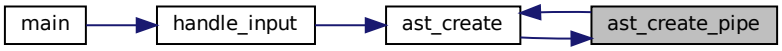
ast_create_pipe - Creates an AST node for a pipe operation. @list: A pointer to the list of token nodes.

This function creates an abstract syntax tree (AST) node for a pipe operation by parsing the token list. It first creates the left node by calling ast_create_redirect. Then, it iterates through the token list to find pipe tokens (TOKEN_↩ PIPE). For each pipe token found, it creates a right node by calling ast_create, and then creates a pipe node with the current token. The left and right nodes are assigned to the pipe node, and the left node is updated to the pipe node. The function returns the final left node, which represents the root of the pipe operation AST.

Return: A pointer to the root AST node of the pipe operation. Here is the call graph for this function:



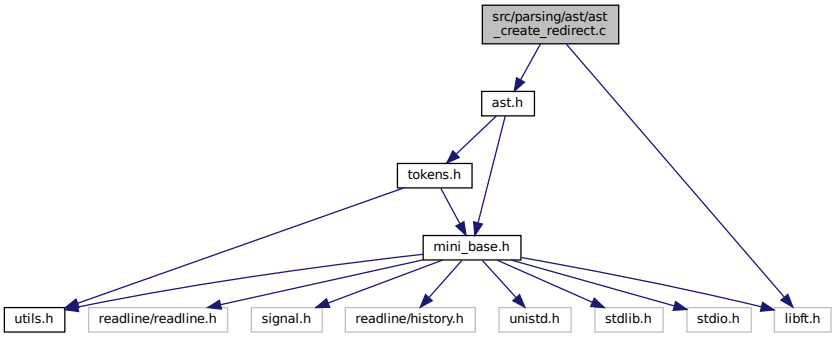
Here is the caller graph for this function:



4.39 src/parsing/ast/ast_create_redirect.c File Reference

```
#include <ast.h>
#include <libft.h>
```

Include dependency graph for ast_create_redirect.c:



Functions

- t_ast_node * ast_create_redirect (t_token_node **list)

4.39.1 Function Documentation

4.39.1.1 ast_create_redirect()

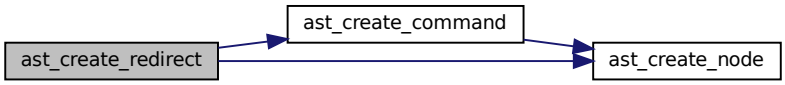
```
t_ast_node* ast_create_redirect (
            t_token_node ** list )
```

ast_create_redirect - Parses a list of tokens to create an abstract syntax tree (AST) node representing a redirection.
@list: A pointer to the list of tokens to be parsed.

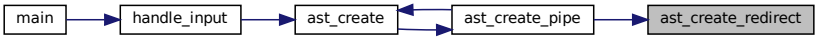
This function parses a list of tokens to create an AST node that represents a redirection operation. It first parses the left-hand side command, then iterates through the list of tokens to check for redirection operators (heredoc, redirect

in, append, redirect out). For each redirection operator found, it creates a new AST node, sets the left and right children, and updates the list pointer to the next token. The function returns the root of the created AST subtree.

Return: A pointer to the root AST node representing the redirection. Here is the call graph for this function:

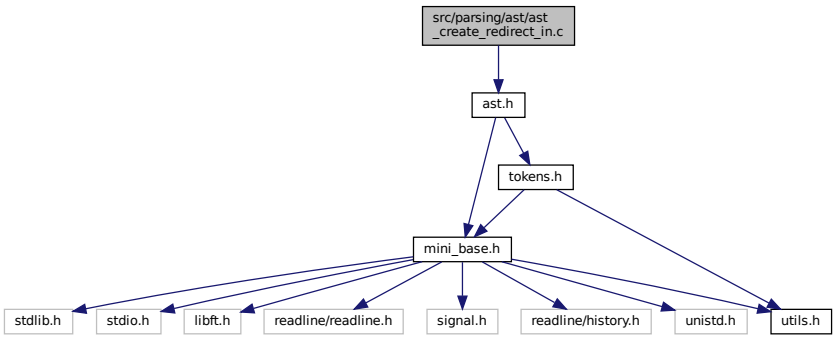


Here is the caller graph for this function:



4.40 src/parsing/ast/ast_create_redirect_in.c File Reference

`#include <ast.h>`
Include dependency graph for ast_create_redirect_in.c:



Functions

- t_ast_node * ast_create_redirect_in_heredoc (t_token_node **list)

4.40.1 Function Documentation

4.40.1.1 ast_create_redirect_in_heredoc()

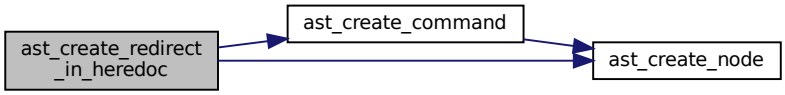
```
t_ast_node* ast_create_redirect_in_heredoc (
            t_token_node ** list )
```

ast_create_redirect_in_heredoc - Parses a list of tokens to create an AST node for input redirection and heredoc.

@list: Double pointer to the head of the token list.

This function processes a list of tokens to create an abstract syntax tree (AST) node for input redirection and heredoc. It starts by creating a command node from the token list and then iterates through the list to handle heredoc and input redirection tokens. For each heredoc or input redirection token, it creates a new AST node, sets the left and right children, and updates the left node to the newly created node. The function returns the final left node, which represents the root of the constructed AST for input redirection and heredoc.

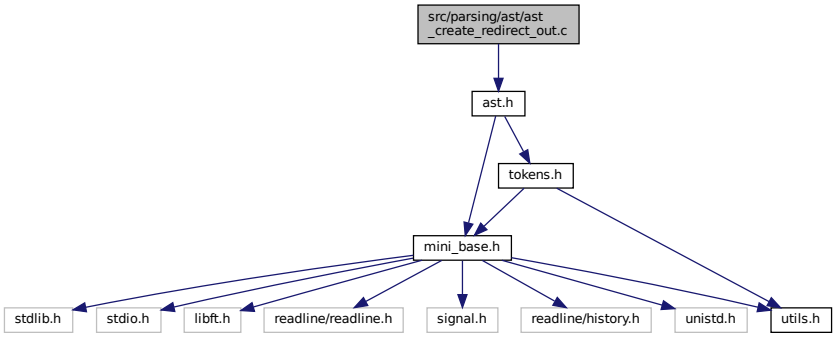
Return: A pointer to the root of the constructed AST node. Here is the call graph for this function:



4.41 src/parsing/ast/ast_create_redirect_out.c File Reference

```
#include <ast.h>
```

Include dependency graph for ast_create_redirect_out.c:



Functions

- t_ast_node * ast_create_redirect_out_append (t_token_node **list)

4.41.1 Function Documentation

4.41.1.1 ast_create_redirect_out_append()

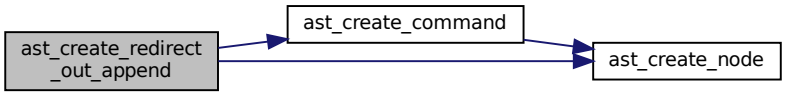
```
t_ast_node* ast_create_redirect_out_append (
            t_token_node ** list )
```

ast_create_redirect_out_append - Parses a list of tokens to create an AST node for redirect out or append operations.

@list: A pointer to the head of the token list.

This function processes a list of tokens to create an abstract syntax tree (AST) node representing redirect out (>) or append (>>) operations. It starts by creating a command node from the token list and then iterates through the list to handle consecutive redirect out or append tokens. For each such token, it creates a new AST node, sets the left and right children, and updates the left child to the newly created node. The function returns the final AST node representing the parsed redirect out or append operations.

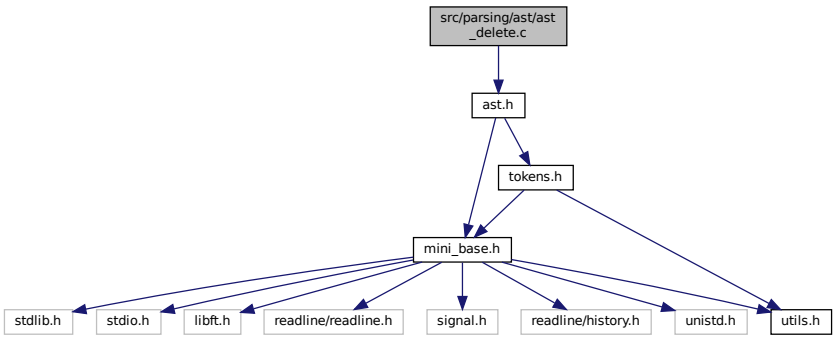
Return: A pointer to the root of the created AST node. Here is the call graph for this function:



4.42 src/parsing/ast/ast_delete.c File Reference

```
#include <ast.h>
```

Include dependency graph for ast_delete.c:



Functions

- void ast_delete (t_ast_node *node)

  Recursively deletes an Abstract Syntax Tree (AST) node and its children.

4.42.1 Function Documentation

4.42.1.1 ast_delete()

```
void ast_delete (
            t_ast_node * node )
```

Recursively deletes an Abstract Syntax Tree (AST) node and its children.

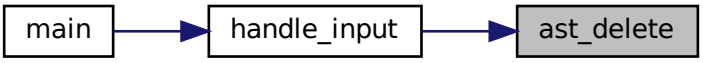
This function takes a pointer to an AST node and recursively deletes its left and right children, as well as the token associated with the node. Finally, it frees the memory allocated for the node itself.

Parameters

node	A pointer to the AST node to be deleted. If the node is NULL, the function returns immediately.

Here is the call graph for this function:

Here is the caller graph for this function:

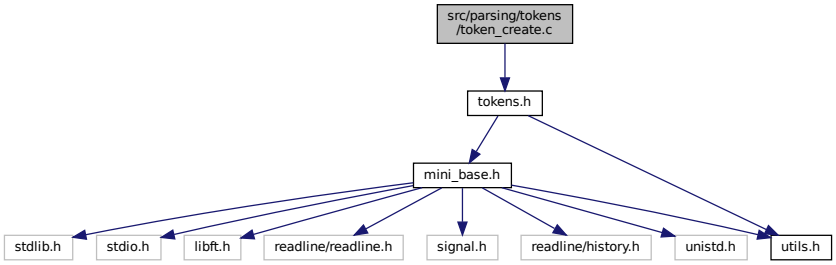


4.43 src/parsing/ast/ast_utils.c File Reference

4.44 src/parsing/tokens/token_create.c File Reference

```
#include <tokens.h>
```

Include dependency graph for token_create.c:



Functions

- t_token_node * token_create (t_token_type type, char *value)

  Creates a new token node with the specified type and value.

4.44.1 Function Documentation

4.44.1.1 token_create()

```
t_token_node* token_create (
            t_token_type type,
            char * value )
```

Creates a new token node with the specified type and value.

This function initializes a new token node using the provided type and value. If the initialization fails, it returns NULL.

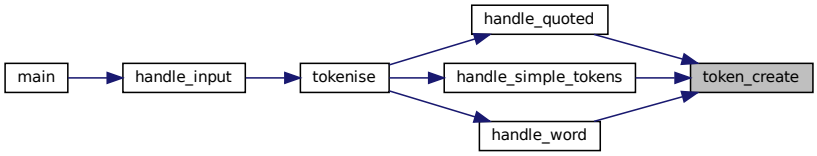
Parameters

type	The type of the token.
value	The value of the token.

Returns

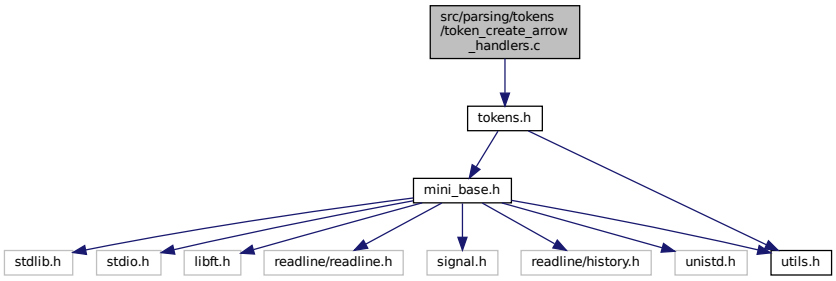
A pointer to the newly created token node, or NULL if initialization fails.

Here is the caller graph for this function:



4.45 src/parsing/tokens/token_create_arrow_handlers.c File Reference

`#include <tokens.h>`
Include dependency graph for token_create_arrow_handlers.c:



Functions

- void handle_arrows (t_token_node **list, char **current)
  Handles the arrows in the input string.

4.45.1 Function Documentation

4.45.1.1 handle_arrows()

```
void handle_arrows (
            t_token_node ** list,
            char ** current )
```

Handles the arrows in the input string.

This function determines whether the current character is '<' or '>'. If it is '<', it calls the handle_arrows_left() function. If it is '>', it calls the handle_arrows_right() function.

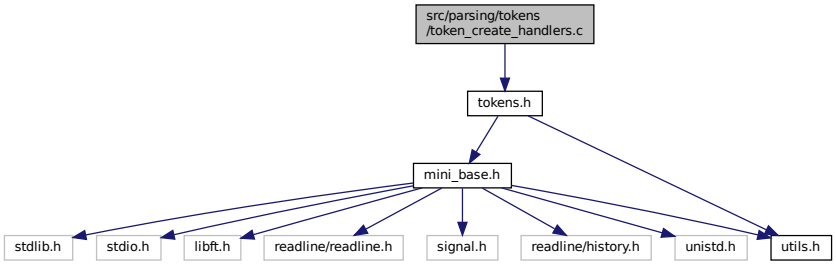
Parameters

current	The current character in the input string.
list	The token list to be updated.

Here is the caller graph for this function:

4.46 src/parsing/tokens/token_create_handlers.c File Reference

`#include <tokens.h>`
Include dependency graph for token_create_handlers.c:



Functions

- void token_append (t_token_node **list, t_token_node *new_node)

  Appends a new token node to the end of the token list.
- void handle_simple_tokens (char **current, t_token_node **list)

  Handles simple tokens in the input string.
- void handle_quoted (char **current, t_token_node **list, char type)

  Handles quoted tokens.
- void handle_word (t_token_node **list, char **current)

  Handles a word in the tokenization process.

4.46.1 Function Documentation

4.46.1.1 handle_quoted()

```
void handle_quoted (
            char ** current,
            t_token_node ** list,
            char type )
```

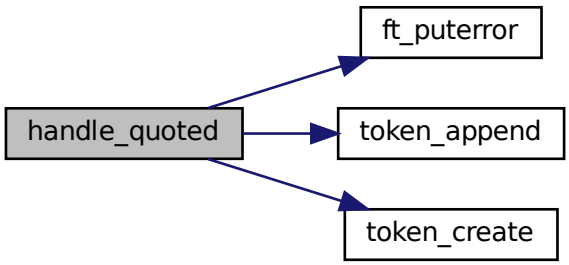
Handles quoted tokens.

This function is responsible for handling quoted tokens in the tokenisation process. It takes a pointer to the current character, a token list, and the type of quote. It searches for the closing quote and creates a token with the quoted content. If the closing quote is missing, it prints an error message.

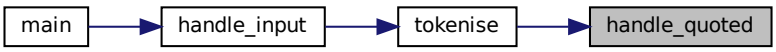
Parameters

current	A pointer to the current character.
list	The token list to add the created token to.
type	The type of quote.

Here is the call graph for this function:



Here is the caller graph for this function:



4.46.1.2 handle_simple_tokens()

```
void handle_simple_tokens (
            char ** current,
            t_token_node ** list )
```

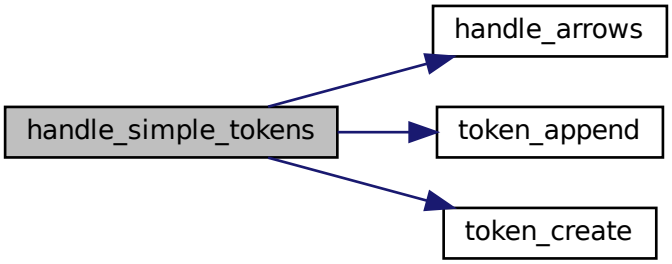
Handles simple tokens in the input string.

This function takes a pointer to the current character in the input string and a pointer to a token list. It checks the current character and adds the corresponding token to the token list. The current character pointer is updated accordingly.

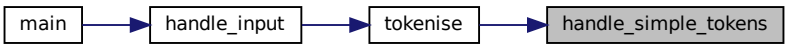
Parameters

current	Pointer to the current character in the input string.
list	Pointer to the token list.

Here is the call graph for this function:



Here is the caller graph for this function:



4.46.1.3 handle_word()

```
void handle_word (
            t_token_node ** list,
            char ** current )
```

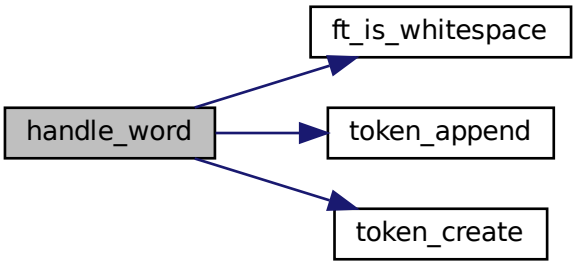
Handles a word in the tokenization process.

This function takes a pointer to the current character and a token list. It iterates through the characters starting from the current position until it reaches the end of the string or encounters a whitespace character, a special character ('&', '|', '(', ')', '<', '>'), or a null character ('\0'). It then adds a token of type TOKEN_WORD to the token list, containing the substring from the start position to the current position.

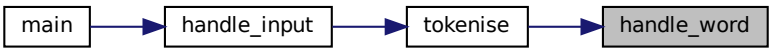
Parameters

current	A pointer to the current character.
list	The token list to add the token to.

Here is the call graph for this function:



Here is the caller graph for this function:



4.46.1.4 token_append()

```
void token_append (
            t_token_node ** list,
            t_token_node * new_node )
```

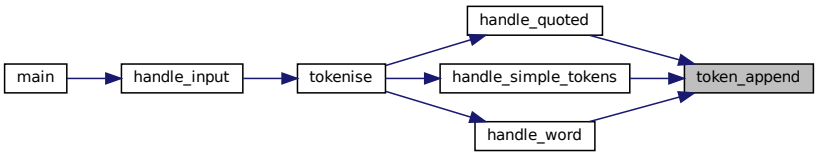
Appends a new token node to the end of the token list.

This function takes a pointer to the head of a token list and a new token node, and appends the new node to the end of the list. If the new node is NULL, the function does nothing. If the list is empty, the new node becomes the head of the list.

Parameters

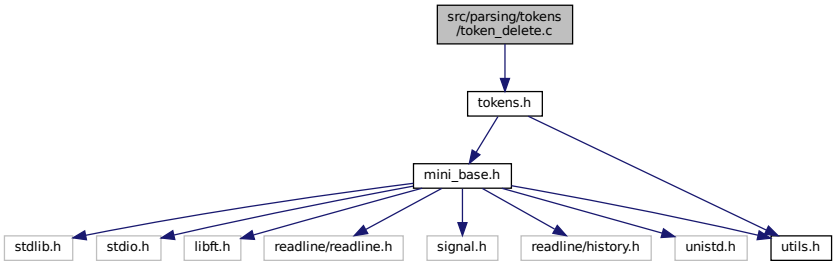
list	A double pointer to the head of the token list.
new_node	A pointer to the new token node to be appended.

Here is the caller graph for this function:



4.47 src/parsing/tokens/token_delete.c File Reference

`#include <tokens.h>`
Include dependency graph for token_delete.c:



Functions

- void token_delete (t_token_node *token)

  Frees the memory allocated for a token and its arguments.
- void token_delete_all (t_token_node **list)

  Frees the memory allocated for a list of tokens.

4.47.1 Function Documentation

4.47.1.1 token_delete()

```
void token_delete (
            t_token_node * token )
```

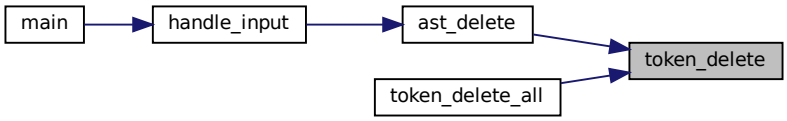
Frees the memory allocated for a token and its arguments.

This function releases the memory allocated for the arguments of a token and then frees the memory allocated for the token itself.

Parameters

token	A pointer to the token to be deleted.

Here is the caller graph for this function:



4.47.1.2 token_delete_all()

```
void token_delete_all (
            t_token_node ** list )
```

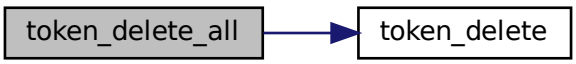
Frees the memory allocated for a list of tokens.

This function iterates through a linked list of tokens, freeing the memory allocated for each token and its arguments. It then frees the memory allocated for the list itself.

Parameters

list	A double pointer to the head of the list of tokens to be deleted.

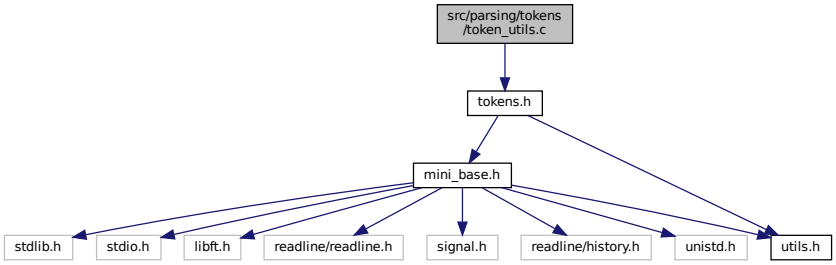
Here is the call graph for this function:



4.48 src/parsing/tokens/token_utils.c File Reference

```
#include <tokens.h>
```

Include dependency graph for token_utils.c:



Functions

- void token_print (t_token_node *token)

  Prints the details of a single token node and its arguments.
- void token_list_print (t_token_node **head)

  Prints the details of a list of token nodes.

4.48.1 Function Documentation

4.48.1.1 token_list_print()

```
void token_list_print (
            t_token_node ** head )
```

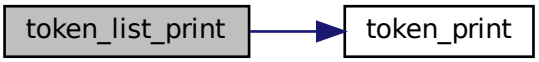
Prints the details of a list of token nodes.

This function checks if the head of the token node list is valid and then calls `token_print` to print the details of the token nodes.

Parameters

head	A double pointer to the head of the token node list.

Here is the call graph for this function:



4.48.1.2 token_print()

```
void token_print (
            t_token_node * token )
```

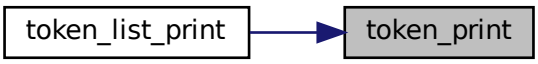
Prints the details of a single token node and its arguments.

This function iterates through a linked list of token nodes and prints the type, command/file, and arguments of each token node. It uses the helper function `_token_type_to_string` to convert the token type to a string for printing.

Parameters

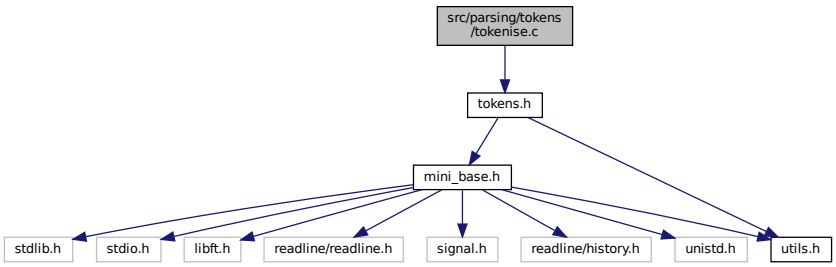
token	A pointer to the first token node in the linked list.

Here is the caller graph for this function:



4.49 src/parsing/tokens/tokenise.c File Reference

`#include <tokens.h>`
Include dependency graph for tokenise.c:



Functions

- t_token_node ** tokenise (char *input)

4.49.1 Function Documentation

4.49.1.1 tokenise()

```
t_token_node** tokenise (
            char * input )
```

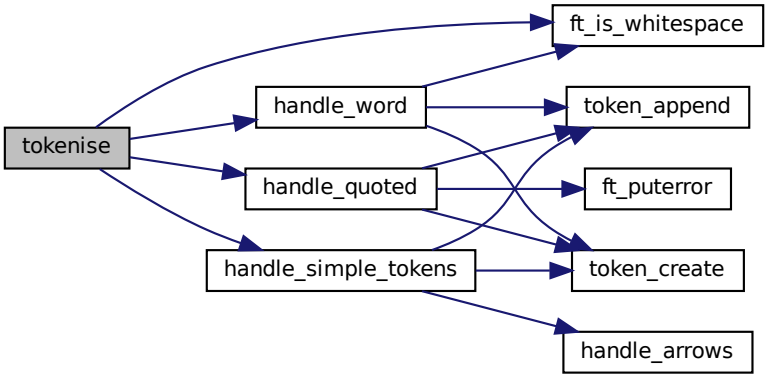
tokenise - Tokenizes the input string into a list of tokens. @input: The input string to be tokenized.

This function takes an input string and tokenizes it into a list of tokens. It handles different types of tokens including simple tokens (like '(', ')', '|', '<', '>'), quoted strings, and words. The function skips over whitespace and processes each token accordingly. The resulting list of tokens is returned.

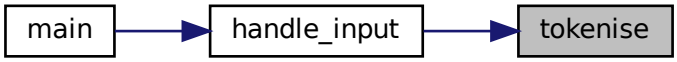
Returns

: A pointer to the list of tokens, or NULL if memory allocation fails.

Here is the call graph for this function:



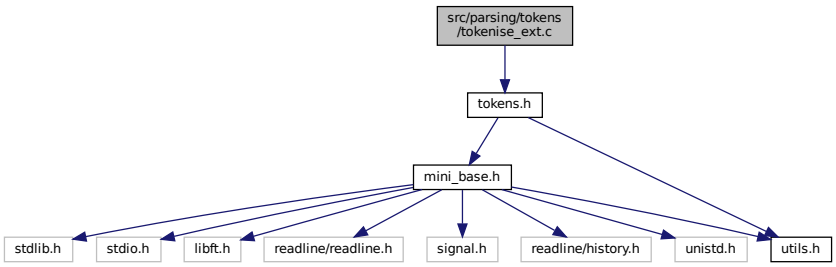
Here is the caller graph for this function:



4.50 src/parsing/tokens/tokenise_ext.c File Reference

```
#include <tokens.h>
```

Include dependency graph for tokenise_ext.c:



Functions

- size_t token_count_args (t_token_node *src)

  Counts the number of consecutive tokens of the same type as the source token.

4.50.1 Function Documentation

4.50.1.1 token_count_args()

```
size_t token_count_args (
            t_token_node * src )
```

Counts the number of consecutive tokens of the same type as the source token.

This function iterates through the linked list of tokens starting from the next token of the given source token (`src`). It counts how many consecutive tokens have the same type as the source token and are of type `TOKEN_STRING`.

Parameters

src	A pointer to the source token node from which to start counting.

Returns

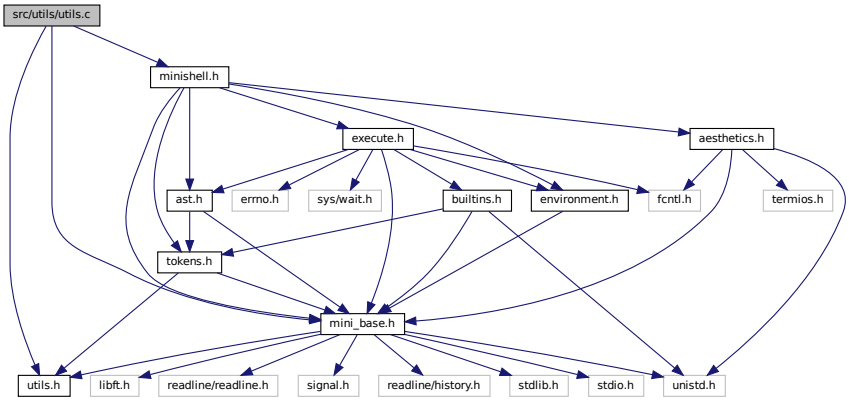
The number of consecutive tokens of the same type as the source token.

4.51 src/utils/utils.c File Reference

```
#include <utils.h>
#include <mini_base.h>
```

```
#include <minishell.h>
```

Include dependency graph for utils.c:



Functions

- int ft_strcmp (const char *s1, const char *s2)

  Compares two null-terminated strings lexicographically.
- t_bool ft_is_whitespace (char c)

  Checks if a character is a whitespace character.
- void ft_puterror (const char *s, char *arg1, const char *arg2)

  Writes an error message to the standard error output.
- void clear_screen_ensure_cursor_visible (void)

  Clears the terminal screen and ensures the cursor is visible at the top-left corner.
- void free_resources (t_shell *shell)

  Frees resources associated with the shell.
- void init_shell (t_shell *shell, const char **envp)

  Initializes the shell structure with environment variables and default settings.
- void handle_input (t_shell *shell)

  Handles the input loop for the shell.

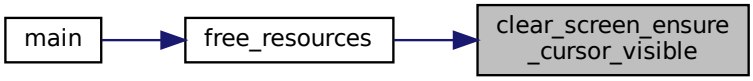
4.51.1 Function Documentation

4.51.1.1 clear_screen_ensure_cursor_visible()

```
void clear_screen_ensure_cursor_visible (
            void  )
```

Clears the terminal screen and ensures the cursor is visible at the top-left corner.

This function sends ANSI escape codes to the terminal to clear the screen and move the cursor to the home position (top-left corner). It is useful for refreshing the terminal display. Here is the caller graph for this function:



4.51.1.2 free_resources()

```
void free_resources (
            t_shell * shell )
```

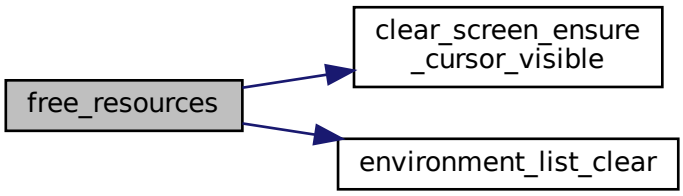
Frees resources associated with the shell.

This function clears the environment list and ensures the screen is cleared and the cursor is visible.

Parameters

shell	A pointer to the shell structure containing resources to be freed.

Here is the call graph for this function:



Here is the caller graph for this function:

4.51.1.3 ft_is_whitespace()

```
t_bool ft_is_whitespace (
            char c )
```

Checks if a character is a whitespace character.

This function checks if the given character c is a whitespace character. Whitespace characters include space (' '), horizontal tab ('\t'), newline ('
'), and carriage return ('\r').

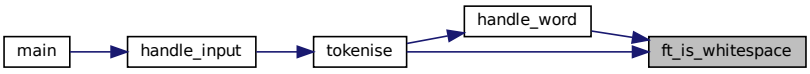
Parameters

c	The character to be checked.

Returns

A boolean value indicating whether the character is a whitespace character.

Here is the caller graph for this function:



4.51.1.4 ft_puterror()

```
void ft_puterror (
            const char * s,
```

```
            char * arg1,
            const char * arg2 )
```

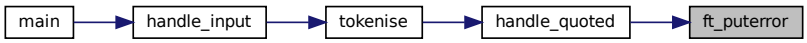
Writes an error message to the standard error output.

This function writes the provided error message and arguments to the standard error output (stderr). If the second argument is NULL, the function returns without writing anything. If the third argument is provided, it will also be written to stderr. A newline character is written at the end of the message.

Parameters

s	The main error message to be written. Must not be NULL.
arg1	The first argument to be written. Must not be NULL.
arg2	The second argument to be written. Can be NULL.

Here is the caller graph for this function:



4.51.1.5 ft_strcmp()

```
int ft_strcmp (
            const char * s1,
            const char * s2 )
```

Compares two null-terminated strings lexicographically.

This function compares the two strings s1 and s2. It returns an integer less than, equal to, or greater than zero if s1 is found, respectively, to be less than, to match, or be greater than s2.

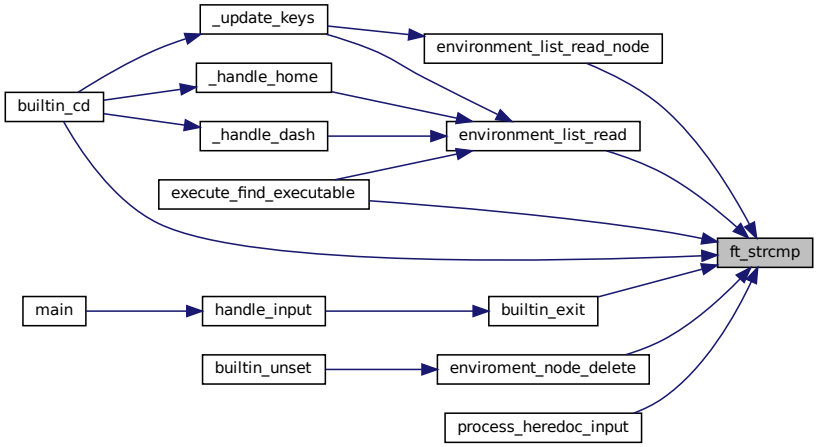
Parameters

s1	The first string to be compared.
s2	The second string to be compared.

Returns

An integer less than, equal to, or greater than zero if s1 is found, respectively, to be less than, to match, or be greater than s2.

Here is the caller graph for this function:



4.51.1.6 handle_input()

```
void handle_input (
            t_shell * shell )
```

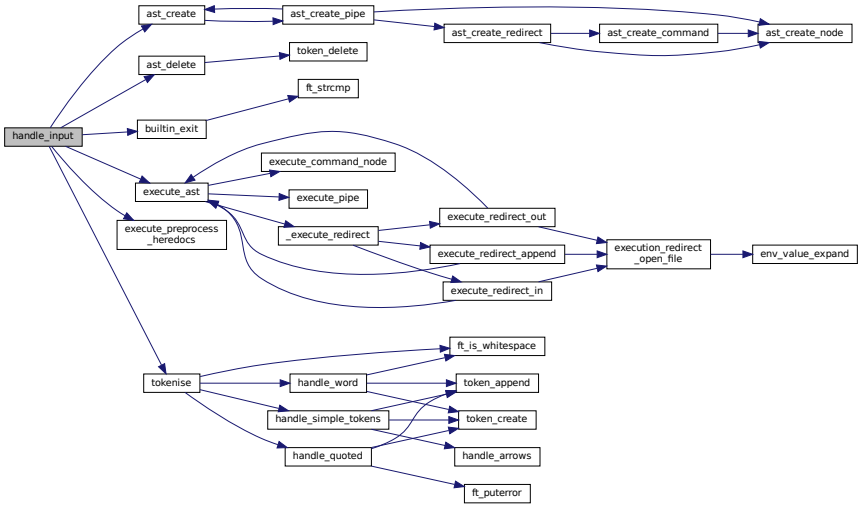
Handles the input loop for the shell.

This function continuously prompts the user for input, tokenizes the input, creates an abstract syntax tree (AST) from the tokens, preprocesses any heredocs, checks for the exit builtin command, executes the AST, and then cleans up the resources.

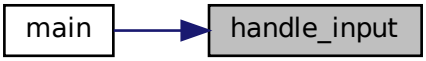
Parameters

shell	A pointer to the shell structure.

Here is the call graph for this function:



Here is the caller graph for this function:



4.51.1.7 init_shell()

```
void init_shell (
            t_shell * shell,
            const char ** envp )
```

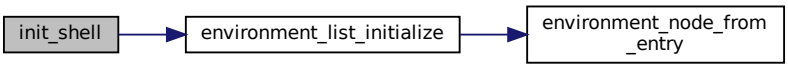
Initializes the shell structure with environment variables and default settings.

This function sets up the initial state of the shell by initializing the environment list, setting the exit code to 0, and disabling top-level redirections. If the environment list initialization fails, the function prints an error message and exits the program.

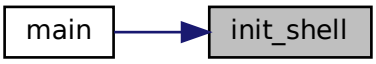
Parameters

shell	Pointer to the shell structure to be initialized.
envp	Array of environment variables.

Here is the call graph for this function:



Here is the caller graph for this function:



Index